论文精读

Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention

Angelos Katharopoulos, Apoorv Vyas,
Nikolaos Pappas, François Fleuret

Idiap Research Institute / EPFL / University of Washington / University of Geneva

ICML 2020

本文档对原论文进行逐段完整翻译、通俗解释、数学推导详解,
并嵌入所有原始图片、表格和算法。

编译方式: `latexmk -xelatex main.tex`

Contents

1 摘要 (Abstract) 2

2 引言 (Introduction) 2

3 相关工作 (Related Work) 4
3.1 高效 Transformer (Efficient Transformers) 4
3.2 理解自注意力 (Understanding Self-Attention) 5
3.3 线性化 softmax (Linearized Softmax) 5

4 线性 Transformer (Linear Transformers) 6
4.1 Transformer 形式化 (§ 3.1) 6
4.2 线性化注意力 (§ 3.2) 7
4.2.1 特征映射与计算代价 (§ 3.2.1) 9
4.3 因果掩码 (§ 3.3) 10
4.3.1 梯度计算 11
4.3.2 训练与推理 12
4.4 Transformer 就是 RNN (§ 3.4) 13

5 实验 (Experiments) 15
5.1 算法：线性 Transformer 因果掩码的前向与反向传播 15
5.2 合成任务 (§ 4.1) 16
5.2.1 收敛性分析 16
5.2.2 内存和计算需求 17
5.3 图像生成 (§ 4.2) 18
5.3.1 MNIST (§ 4.2.1) 18
5.3.2 CIFAR-10 (§ 4.2.2) 20
5.4 自动语音识别 (§ 4.3) 21

6 结论 (Conclusions) 22

7 致谢 (Acknowledgements) 23

A 附录 A：梯度推导 (Gradient Derivation) 23

B 附录 B：训练演化 (Training Evolution) 25

C 附录 C：图像生成吞吐量讨论 26
C.1 有状态 softmax 注意力 (Stateful Softmax Attention) 26
C.2 等化批大小 (Equalizing the Batch Size) 27

D 附录 D：图像生成定性结果 28

E 附录 E：参考文献一览表 32

1 摘要 (Abstract)

原文翻译

Transformer 在多个任务上取得了卓越的性能，但由于其相对于输入长度的二次复杂度，对于非常长的序列来说速度极慢。为了解决这一限制，我们将自注意力表示为核特征映射的线性点积，并利用矩阵乘积的结合律将复杂度从 $\mathcal{O}\left(N^2\right)$ 降低到 $\mathcal{O}(N)$，其中 N 是序列长度。我们表明，这种表述允许一种迭代实现，可以极大地加速自回归 Transformer，并揭示了它们与循环神经网络之间的关系。我们的**线性 Transformer**（linear transformer）达到了与标准 Transformer 相似的性能，并且在非常长序列的自回归预测上快了高达 4000 倍。

通俗解释

想象你在一个大会议室里，有 N 个人，每个人都需要和其他所有人交流一次才能做出决定。标准 Transformer 就是这样工作的——每个“词”都要和序列中的每个其他“词”计算注意力，所以计算量是 $N \times N = N^2$。
这篇论文提出了一个巧妙的数学技巧：把注意力计算重新排列（利用矩阵乘法结合律），使得不需要每个人都跟所有人单独说话，而是大家把信息汇总到一个“公告板”上，每个人只需要看一次公告板即可。这把计算量从 N^2 降到了 N。
更厉害的是，当做自回归生成（比如一个一个像素生成图片时），这种线性 Transformer 本质上等价于一个 RNN（循环神经网络），每次只需要更新一个固定大小的“记忆状态”，不需要回头看所有历史信息，因此推理速度可以快上千倍。

关键概念

核心贡献有三点：（1）用核函数特征映射替代 softmax，把注意力线性化；（2）利用矩阵乘法结合律把复杂度从 $\mathcal{O}\left(N^2\right)$ 降到 $\mathcal{O}(N)$；（3）揭示了线性注意力 Transformer 等价于 RNN，使自回归推理具有常数内存和常数时间步代价。

2 引言 (Introduction)

原文翻译

Transformer 模型最初由 Vaswani 等人（2017）[a] 在神经机器翻译的背景下提出【Sutskever et al. 2014：序列到序列学习的开创性工作；Bahdanau et al. 2015：引入注意力机制的翻译模型。在此引用是因为 Transformer 的诞生背景就是机器翻译】，并在处理自然语言【Devlin et al. 2019（BERT）：基于 Transformer 的双向预训练模型，此处说明 Transformer 在 NLP 中的广泛成功】、音频【Sperber et al. 2018：将自注意力用于声学模型】和图像【Parmar et al. 2019：将独立的自注意力用于视觉模型】的各种任务上展示了令人印象深刻的结果。除了有大量监督数据的任务之外，Transformer 在通过自回归【Radford et al. 2018（GPT-1），Radford et al. 2019（GPT-2）：生成式预训练语言模型】或掩码语言建模目标【Devlin et al. 2019（BERT）；Yang et al. 2019（XLNet）；Song et al. 2019（MASS）；Liu et al. 2020（RoBERTa）：各种掩码/自回归预训练方法】进行预训练时，也能有效地将知识迁移到监督有限或无监督的任务中。
然而，这些好处通常伴随着非常高的计算和内存成本。瓶颈主要由自注意力的全局感受野造成，它以 $\mathcal{O}\left(N^2\right)$ 的二次内存和时间复杂度处理 N 个输入的上下文。因此，在实践中 Transformer 训练缓慢且其上下文是**有限的**。这破坏了时间连贯性并阻碍了捕获长期

依赖关系。Dai 等人（2019）【Dai et al. 2019（Transformer-XL）：通过关注之前上下文的记忆来处理超长依赖，但增加了计算开销】通过关注之前上下文的记忆来解决这一问题，但代价是计算效率的下降。
近来，研究者将注意力转向在不牺牲效率的情况下增加上下文长度的方法。为此，Child 等人（2019）【Child et al. 2019（Sparse Transformer）：用稀疏因式分解降低注意力矩阵计算量到 $\mathcal{O}\left(N\sqrt{N}\right)$】引入了注意力矩阵的稀疏因式分解，将自注意力复杂度降低到 $\mathcal{O}\left(N\sqrt{N}\right)$。Kitaev 等人（2020）【Kitaev et al. 2020（Reformer）：用局部敏感哈希进一步降低到 $\mathcal{O}(N\log N)$】使用局部敏感哈希（LSH）进一步将复杂度降低到 $\mathcal{O}(N\log N)$。这使得扩展到长序列成为可能。尽管上述模型可以在大序列上高效训练，但它们**并不加速自回归推理**。
在本文中，我们引入了**线性 Transformer**模型，该模型显著减少了内存占用并相对于上下文长度线性扩展。我们通过使用基于核的自注意力表述和矩阵乘积的结合律来计算自注意力权重（§ 3.2）。利用我们的线性表述，我们还以线性复杂度和常数内存表达了因果掩码（§ 3.3）。这揭示了 Transformer 和 RNN 之间的关系，使我们能够以快几个数量级的速度执行自回归推理（§ 3.4）。
我们在图像生成和自动语音识别上的评估表明，线性 Transformer 可以达到标准 Transformer 的性能水平，同时在推理期间快高达三个数量级。

[a] Vaswani et al.，“Attention Is All You Need”，NIPS 2017.

通俗解释

引言部分讲了三件事：
1. Transformer 很强但很慢：Transformer 自 2017 年问世以来横扫了 NLP、语音、图像等多个领域，但它的核心操作——自注意力——需要让序列中的每个位置都和其他所有位置交互，计算量随序列长度呈二次增长。假设序列有 1000 个 token，就要做 $1000 \times 1000 = 100$ 万次比较；如果是 10000 个 token，就是 1 亿次。
2. 已有的加速方案不够好：Sparse Transformer 和 Reformer 分别把复杂度降到了 $\mathcal{O}\left(N\sqrt{N}\right)$ 和 $\mathcal{O}(N\log N)$，但在自回归任务（一个接一个生成 token）中依然没有本质加速，因为每生成一个新 token 都要重新计算注意力。
3. 本文的解决方案：通过把 softmax 注意力替换为核函数的特征映射，再利用一个简单的数学性质（矩阵乘法结合律），把复杂度降到 $\mathcal{O}(N)$。更关键的是，这使得 Transformer 可以像 RNN 一样迭代运行，自回归推理快了上千倍。

注意事项

本文提出的加速主要体现在**自回归推理**（逐步生成）阶段，而非训练阶段。训练时标准 Transformer 可以并行计算所有位置的注意力，线性 Transformer 的训练加速更多体现在内存节省和长序列处理上。另外，已有方法如 Reformer 虽然训练时复杂度低，但自回归推理时仍然慢——本文正是解决这个痛点。

3 相关工作 (Related Work)

原文翻译

在本节中，我们概述了最相关的工作，这些工作寻求解决 Transformer 的大内存和高计算需求。此外，我们讨论了从理论上分析 Transformer 模型核心组件（即自注意力）的方法。最后，我们介绍了另一条寻求缓解注意力计算中 softmax 瓶颈的工作。

3.1 高效 Transformer (Efficient Transformers)

原文翻译

现有工作寻求通过权重剪枝 【Michel et al. 2019：证明很多注意力头可以被剪枝，16 个头未必都需要】、 权重分解 【Lan et al. 2020（ALBERT）：通过参数共享和嵌入分解减小模型大小】、 权重量化 【Zafrir et al. 2019（Q8BERT）：8 位量化 BERT】 或知识蒸馏来提高 Transformer 的内存效率。Clark 等人（2020） 【Clark et al. 2020（ELECTRA）：提出替代 token 检测预训练目标，更样本高效】 提出了一种称为替代 token 检测的新预训练目标，样本效率更高，减少了整体计算。Lample 等人（2019） 【Lample et al. 2019：使用乘积键注意力增加层的容量，计算开销可忽略】 使用乘积键注意力来增加任何层的容量，计算开销可忽略不计。
使用这些方法减少内存或计算需求可以带来训练或推理时间的加速，但从根本上说，时间复杂度相对于序列长度仍然是二次的，这阻碍了扩展到长序列。相比之下，我们表明我们的方法在理论上（§ 3.2）和经验上（§ 4.1）都降低了 Transformer 的内存和时间复杂度。
另一条研究旨在增加 Transformer 中自注意力的“上下文”。上下文指的是用于计算自注意力的序列的最大部分。Dai 等人（2019） 【Dai et al. 2019（Transformer-XL）：通过维护前一上下文的记忆来学习超越固定长度的依赖】 引入了 Transformer-XL，通过学习超越固定长度上下文的依赖关系，在语言建模中达到了最先进水平，而不破坏时间连贯性。然而，在内存中维护先前的上下文引入了显著的额外计算成本。相比之下，Sukhbaatar 等人（2019） 【Sukhbaatar et al. 2019：通过学习每个注意力头的最优注意力跨度来显著扩展上下文长度】 通过学习每个注意力头的最优注意力跨度显著扩展了上下文长度，同时保持对内存占用和计算时间的控制。注意，这两种方法与标准模型具有相同的渐近复杂度。相比之下，我们改进了自注意力的渐近复杂度，这使我们能够使用显著更大的上下文。
与我们的模型更相关的是 Child 等人（2019）和 Kitaev 等人（2020）的工作。前者引入了注意力矩阵的稀疏因式分解，将整体复杂度从二次降低到 $\mathcal{O}\left(N\sqrt{N}\right)$，用于长序列的生成建模。最近，Kitaev 等人（2020）提出了 Reformer。该方法通过使用局部敏感哈希（LSH）执行更少的点积运算，进一步将复杂度降低到 $\mathcal{O}(N \log N)$。注意，为了能够使用 LSH，Reformer 将注意力的键约束为与查询相同。因此，该方法不能用于键需要与查询不同的解码任务。相比之下，线性 Transformer 不对查询和键施加任何约束，并且相对于序列长度线性扩展。此外，它们可以用于在自回归任务中以快三个数量级的速度执行推理，同时在验证困惑度方面达到可比的性能。

通俗解释

相关工作按三个维度对比了已有加速方案：
模型压缩类（剪枝、量化、蒸馏、参数共享）：这些方法减小了模型大小或计算量，但注意力计算本身仍然是 N^2 的，没有解决根本问题。
扩展上下文类（Transformer-XL、自适应注意力跨度）：允许模型“看到”更远的历

史，但渐近复杂度没有改变。
降低注意力复杂度类（Sparse Transformer $\mathcal{O}\left(N\sqrt{N}\right)$, Reformer $\mathcal{O}(N\log N)$）：这些最接近本文的工作。但 Reformer 有一个关键限制——它要求 Key = Query（因为 LSH 需要比较同一空间中的向量），所以在解码任务中无法使用。本文的线性 Transformer 没有这个限制，且实现了更优的 $\mathcal{O}(N)$。

3.2 理解自注意力 (Understanding Self-Attention)

原文翻译

从理论角度更好地理解自注意力的努力并不多。Tsai 等人（2019）【Tsai et al. 2019：提出注意力的核函数视角，将注意力视为对输入施加核平滑器，核分数是输入之间的相似度。本文正是沿用该核函数视角来加速注意力计算】提出了 Transformer 中注意力的基于核的表述，将注意力视为对输入施加核平滑器（kernel smoother），核分数是输入之间的相似度。该表述提供了一种更好的方式来理解注意力组件并集成位置嵌入。相比之下，我们使用核表述来加速自注意力的计算并降低其计算复杂度。另外，我们观察到如果对查询和键应用具有正相似度分数的核，线性注意力可以正常收敛。
最近，Cordonnier 等人（2020）【Cordonnier et al. 2020：证明具有足够多头的多头自注意力可以表达任何卷积层。本文则是揭示 Transformer 与 RNN 的关系】提供了理论证明和实验证据，表明具有足够数量头的多头自注意力可以表达任何卷积层。而在本文中，我们则表明用自回归目标训练的自注意力层可以被视为循环神经网络，这一观察可以用来显著加速自回归 Transformer 模型的推理时间。

3.3 线性化 softmax (Linearized Softmax)

原文翻译

多年来，softmax 一直是训练具有大量类别的分类模型的瓶颈【Goodman 2001；Morin & Bengio 2005；Mnih & Hinton 2009：各种加速大类别 softmax 的方法（层次softmax等）】。最近的工作【Blanc & Rendle 2017；Rawat et al. 2019：用特征映射的线性点积近似 softmax 以加速训练】用特征映射的线性点积来近似 softmax，通过采样加速训练。受这些工作的启发，我们线性化了 Transformer 中的 softmax 注意力。与本工作同期，Shen 等人（2020）【Shen et al. 2020：也探索了线性化注意力，但仅用于目标检测。本文不仅线性化注意力，还开发了线性复杂度常数内存的自回归模型】探索了线性化注意力在图像目标检测任务中的使用。相比之下，我们不仅线性化了注意力计算，还开发了一种具有线性复杂度和常数内存的自回归 Transformer 模型，用于推理和训练。此外，我们表明通过核的视角，每个 Transformer 都可以被视为一个循环神经网络。

第2节总结：相关工作定位

本文的独特贡献在于三个“不仅仅”：

1. 不仅仅是模型压缩——而是从根本上改变了注意力的复杂度；
2. 不仅仅是训练加速——更关键的是自回归推理的千倍加速；
3. 不仅仅是线性化注意力——还揭示了 Transformer = RNN 的等价关系。

4 线性 Transformer (Linear Transformers)

原文翻译

在本节中，我们正式介绍我们提出的线性 Transformer。我们展示将注意力从传统的 softmax 注意力更改为基于特征映射的点积注意力，可以带来更好的时间和内存复杂度，以及一个可以以线性时间执行序列生成的因果模型，类似于循环神经网络。
首先，在 § 3.1 中，我们介绍 Vaswani 等人（2017）引入的 Transformer 架构的形式化。随后，在 § 3.2 和 § 3.3 中，我们介绍我们提出的线性 Transformer。最后，在 § 3.4 中，我们将 Transformer 重写为循环神经网络。

4.1 Transformer 形式化 (§ 3.1)

原文翻译

设 $x \in \mathbb{R}^{N\times F}$ 表示一个由 N 个 F 维特征向量组成的序列。Transformer 是一个函数 $T : \mathbb{R}^{N\times F} \to \mathbb{R}^{N\times F}$，定义为 L 个 Transformer 层 $T_1(\cdot), \ldots, T_L(\cdot)$ 的组合，如下：

$$T_l(x) = f_l(A_l(x) + x). \tag{1}$$

函数 $f_l(\cdot)$ 独立地变换每个特征，通常用一个小的两层前馈网络实现。$A_l(\cdot)$ 是自注意力函数，是 Transformer 中唯一跨序列位置起作用的部分。
自注意力函数 $A_l(\cdot)$ 为每个位置计算所有其他位置的特征表示的加权平均，权重与表示之间的相似度分数成正比。形式上，输入序列 x 被三个矩阵 $W_Q \in \mathbb{R}^{F\times D}$、$W_K \in \mathbb{R}^{F\times D}$ 和 $W_V \in \mathbb{R}^{F\times M}$ 投影到对应的表示 Q、K 和 V。所有位置的输出 $A_l(x) = V'$ 的计算如下：

$$\begin{aligned} Q &= xW_Q, \\ K &= xW_K, \\ V &= xW_V, \\ V' &= \mathrm{softmax}\left(\frac{QK^T}{\sqrt{D}}\right) V. \end{aligned} \tag{2}$$

注意在上式中，softmax 函数是对 QK^T 逐行应用的。按照通常的术语，Q、K 和 V 分别被称为“查询”（queries）、“键”（keys）和“值”（values）。
公式 2 实现了一种特定形式的自注意力，称为 softmax 注意力，其中相似度分数是查询和键的点积的指数。给定用下标 i 对矩阵取第 i 行作为向量的约定，我们可以为任意相似度函数写出广义注意力公式如下：

$$V'_i = \frac{\sum_{j=1}^{N} \mathrm{sim}\left(Q_i, K_j\right) V_j}{\sum_{j=1}^{N} \mathrm{sim}\left(Q_i, K_j\right)}. \tag{3}$$

如果我们将相似度函数替换为 $\mathrm{sim}\left(q, k\right) = \exp\left(\frac{q^T k}{\sqrt{D}}\right)$，则公式 3 等价于公式 2。

通俗解释

这部分用数学语言精确定义了标准 Transformer 的结构：
一个 Transformer 层干了什么? 把输入 x 先通过自注意力 $A_l(x)$ 做“信息交流”（每个位置都看看其他位置），然后加上残差连接 $+x$，再通过一个前馈网络 f_l。
自注意力怎么算? 把 x 分别投影成 Q（查询）、K（键）、V（值），然后计算

QK^T（所有查询和所有键的相似度矩阵，$N \times N$ 大小），过 softmax 归一化，再乘以 V 得到输出。
广义注意力： softmax 注意力只是“相似度函数取为指数点积”的特例。可以用任何非负的相似度函数替代，只要满足可以归一化即可。这为后面用核函数替代打下了基础。

公式 2 维度分析与数值例子

假设 $N = 3$（序列长度3），$F = 4$（特征维度4），$D = 2$（Q/K 维度），$M = 2$（V 维度）。
维度链路：

- $x \in \mathbb{R}^{3\times4}$，$W_Q \in \mathbb{R}^{4\times2}$，所以 $Q = xW_Q \in \mathbb{R}^{3\times2}$
- 同理，$K \in \mathbb{R}^{3\times2}$，$V \in \mathbb{R}^{3\times2}$
- $QK^T \in \mathbb{R}^{3\times2} \cdot \mathbb{R}^{2\times3} = \mathbb{R}^{3\times3}$——这就是 $N \times N$ 的注意力矩阵
- $\text{softmax}(QK^T/\sqrt{D}) \in \mathbb{R}^{3\times3}$（逐行 softmax）
- $V' = \text{softmax}(\cdots) \cdot V \in \mathbb{R}^{3\times3} \cdot \mathbb{R}^{3\times2} = \mathbb{R}^{3\times2}$

数值例子： 假设 $Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$，$K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{pmatrix}$，$D = 2$。

$$QK^T/\sqrt{2} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 0.5 \\ 0 & 1 & 0.5 \\ 1 & 1 & 1 \end{pmatrix} \approx \begin{pmatrix} 0.707 & 0 & 0.354 \\ 0 & 0.707 & 0.354 \\ 0.707 & 0.707 & 0.707 \end{pmatrix}$$

对第一行做 softmax：$\text{softmax}(0.707, 0, 0.354) = \frac{(e^{0.707}, e^0, e^{0.354})}{e^{0.707}+e^0+e^{0.354}} = \frac{(2.028, 1, 1.425)}{4.453} \approx (0.455, 0.225, 0.320)$。
可以看到：位置1 对自己（K_1）的注意力权重最高（0.455），对位置3（K_3）中等（0.320），对位置2（K_2）最低（0.225）——因为 Q_1 和 K_1 最相似。
关键观察： 计算 QK^T 的代价是 $\mathcal{O}(N^2D)$，存储注意力矩阵的代价是 $\mathcal{O}(N^2)$——这就是二次复杂度的来源。

4.2 线性化注意力（§ 3.2）

原文翻译

公式 2 中注意力的定义是通用的，可以用来定义其他几种注意力实现，如多项式注意力或 RBF 核注意力【Tsai et al. 2019：从核函数视角分析注意力，说明多种核都可以替代 softmax】。注意，为使公式 3 定义一个注意力函数，我们对 $\text{sim}(\cdot)$ 施加的唯一约束是它必须非负。这包括所有核 $k(x, y) : \mathbb{R}^{2\times F} \to \mathbb{R}_+$。
给定这样一个具有特征表示 $\phi(x)$ 的核，我们可以将公式 2 重写为：

$$V_i' = \frac{\sum_{j=1}^{N} \phi(Q_i)^T \phi(K_j) V_j}{\sum_{j=1}^{N} \phi(Q_i)^T \phi(K_j)}, \tag{4}$$

然后通过利用矩阵乘法的结合律进一步简化为：

$$V_i' = \frac{\phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j)}. \tag{5}$$

上式的分子以向量化形式写出更容易理解：

$$\left(\phi(Q)\phi(K)^T\right)V = \phi(Q)\left(\phi(K)^T V\right). \tag{6}$$

注意特征映射 $\phi(\cdot)$ 是对矩阵 Q 和 K 逐行应用的。
从公式 2 可以看出，softmax 注意力的计算成本随 $\mathcal{O}(N^2)$ 增长，其中 N 是序列长度。内存需求也是如此，因为必须存储完整的注意力矩阵以计算关于查询、键和值的梯度。相比之下，我们提出的线性 Transformer（公式 5）具有 $\mathcal{O}(N)$ 的时间和内存复杂度，因为我们可以计算 $\sum_{j=1}^{N}\phi(K_j)V_j^T$ 和 $\sum_{j=1}^{N}\phi(K_j)$ 一次，然后为每个查询重复使用。

通俗解释

这是全文最核心的技巧，让我用一个类比来解释：
标准注意力就像点名提问：老师（查询 Q_i）要依次问教室里的每个学生（键 K_j）一个问题，根据回答（值 V_j）综合信息。有 N 个老师和 N 个学生，总共要进行 N^2 次对话。
线性注意力就像先汇总再查阅：如果把所有学生的信息先汇总到一块黑板上（$\sum_j \phi(K_j)V_j^T$，这是一个固定大小的矩阵），那么每个老师只需要看一次黑板即可。汇总一次花 N 步，查阅 N 次每次常数时间，总共 $\mathcal{O}(N)$。
数学上，关键在于：$\phi(Q_i)^T\phi(K_j)$ 是一个标量（两个向量的点积），可以拆成 $\phi(Q_i)^T$ 和 $\phi(K_j)$ 两部分。由于 $\phi(Q_i)^T$ 对所有 j 都一样，所以可以提到求和符号外面。这就是矩阵乘法结合律的威力：
$(AB)C$ 中 A 是 $N\times N$ 的注意力矩阵（代价 N^2），但如果改成 $A(BC)$，BC 是一个小矩阵（代价 ND），$A(BC)$ 的代价也只是 ND，总共 $\mathcal{O}(ND)$——线性！

核心技巧：结合律变换的推导

出发点：公式 4 的分子为

$$\text{Num}_i = \sum_{j=1}^{N}\underbrace{\phi(Q_i)^T\phi(K_j)}_{\text{标量}}\cdot V_j$$

其中 $\phi(Q_i)\in\mathbb{R}^C$，$\phi(K_j)\in\mathbb{R}^C$，$V_j\in\mathbb{R}^M$。
第一步：将标量乘以向量改写为外积形式。注意 $\phi(Q_i)^T\phi(K_j)$ 是标量，所以：

$$\phi(Q_i)^T\phi(K_j)\cdot V_j = \phi(Q_i)^T\left(\phi(K_j)V_j^T\right)^T$$

但更直接的做法是利用：$a^Tb\cdot v = a^T(bv^T)$，其中 $a,b\in\mathbb{R}^C, v\in\mathbb{R}^M$，$bv^T\in\mathbb{R}^{C\times M}$。
第二步：求和后提出 $\phi(Q_i)^T$：

$$\text{Num}_i = \sum_{j=1}^{N}\phi(Q_i)^T\left(\phi(K_j)V_j^T\right) = \phi(Q_i)^T\underbrace{\left(\sum_{j=1}^{N}\phi(K_j)V_j^T\right)}_{S\in\mathbb{R}^{C\times M}}$$

关键：$S=\sum_{j=1}^{N}\phi(K_j)V_j^T$ 只需要计算一次（$\mathcal{O}(NCM)$），然后对每个 i 计算 $\phi(Q_i)^T S$ 只需 $\mathcal{O}(CM)$，总共 $\mathcal{O}(NCM)$。
对比：标准注意力先算 $\phi(Q)\phi(K)^T$（$N\times N$ 矩阵，$\mathcal{O}(N^2C)$），再乘 V（$\mathcal{O}(N^2M)$）。当 $N\gg C,M$ 时，线性注意力的优势巨大。
数值例子：设 $N=1000, C=64, M=64$：标准注意力：$N^2\cdot C = 1000^2\cdot 64 = 6.4\times 10^7$，线性注意力：$N\cdot C\cdot M = 1000\cdot 64\cdot 64 = 4.1\times 10^6$，快约 16 倍。当 $N=10000$ 时，标准注意力 $=6.4\times 10^9$，线性注意力 $=4.1\times 10^7$，快约 156 倍。

注意事项

为什么不能直接对 softmax 注意力使用这个技巧? 因为 softmax 注意力的相似度函数是 $\exp(q^T k/\sqrt{D})$，其对应的核的特征映射是无穷维的（高斯核/指数核的特征空间是无穷维 Hilbert 空间）。所以不可能找到一个有限维的 ϕ 使得 $\exp(q^T k) = \phi(q)^T \phi(k)$。本文的做法是换一个核（换一个 ϕ），而不是精确近似 softmax。

4.2.1 特征映射与计算代价（§ 3.2.1）

原文翻译

对于 softmax 注意力，乘法和加法的总代价为 $\mathcal{O}\left(N^2 \max(D, M)\right)$，其中 D 是查询和键的维度，M 是值的维度。相反，对于线性注意力，我们首先计算维度为 C 的特征映射。随后，计算新值需要 $\mathcal{O}(NCM)$ 次加法和乘法。
之前的分析没有考虑核和特征函数的选择。注意，对应于指数核的特征函数是无穷维的，这使得精确 softmax 注意力的线性化不可行。另一方面，多项式核有精确的有限维特征映射，并且已被证明与指数或 RBF 核效果一样好。2 次多项式线性化 Transformer 的计算代价是 $\mathcal{O}\left(ND^2M\right)$。这使得当 $N > D^2$ 时计算复杂度有利。注意在实践中这是成立的，因为我们希望能处理数万个元素的序列。
对于我们的实验（处理较小的序列），我们采用如下产生正相似度函数的特征映射：

$$\phi(x) = \mathrm{elu}(x) + 1, \tag{7}$$

其中 $\mathrm{elu}(\cdot)$ 是指数线性单元激活函数【Clevert et al. 2015：提出 ELU 激活函数】。我们选择 $\mathrm{elu}(\cdot)$ 而非 $\mathrm{relu}(\cdot)$，以避免在 x 为负时将梯度设为 0。该特征映射产生的注意力函数需要 $\mathcal{O}(NDM)$ 次乘法和加法。在实验部分，我们展示公式 7 的特征映射在性能上与完整 Transformer 持平，同时显著降低了计算和内存需求。

通俗解释

这里讨论了特征映射 ϕ 的选择问题：
理想但不可行： 如果能找到有限维的 ϕ 使得 $\phi(q)^T\phi(k) = \exp(q^T k/\sqrt{D})$，那就完美了——既保留 softmax 的效果又获得线性复杂度。但这个 ϕ 是无穷维的，做不到。
多项式核： 二次多项式核 $k(x, y) = (x^T y)^2$ 有精确的有限维特征映射（所有二次单项式），维度是 D^2。代价 $\mathcal{O}\left(ND^2M\right)$，当序列很长（$N > D^2$）时优于 $\mathcal{O}\left(N^2\right)$。
实际选择——elu+1： 对于本文的实验，作者选了一个更简单的特征映射 $\phi(x) = \mathrm{elu}(x) + 1$。这个映射保证输出非负（因为 $\mathrm{elu}(x) + 1 > 0$ 恒成立），维度不变（$C = D$），所以代价是 $\mathcal{O}(NDM)$，非常高效。选 elu 而不是 relu 是因为 relu 在负值区域梯度为零，会导致训练问题。

特征映射 $\phi(x) = \mathrm{elu}(x) + 1$ 的性质

ELU 的定义是：

$$\mathrm{elu}(x) = \begin{cases} x & \text{if } x > 0 \\ e^x - 1 & \text{if } x \le 0 \end{cases}$$

所以 $\phi(x) = \mathrm{elu}(x) + 1 = \begin{cases} x + 1 & \text{if } x > 0 \\ e^x & \text{if } x \le 0 \end{cases}$

非负性验证： 当 $x > 0$，$\phi(x) = x + 1 > 1 > 0$；当 $x \le 0$，$\phi(x) = e^x \in (0, 1]$。所以 $\phi(x) > 0$ 恒成立。

为何非负很重要？ 注意力权重（softmax 输出）必须非负才能解释为“加权平均”。如果 ϕ 可以产生负值，则 $\phi(Q_i)^T\phi(K_j)$ 可能为负，归一化后的“注意力权重”失去概率含义，可能导致不稳定。

梯度： $\phi'(x) = \begin{cases} 1 & \text{if } x > 0 \\ e^x & \text{if } x \leq 0 \end{cases}$ 当 x 很负时，梯度 $e^x \to 0$ 但不完全为 0（对比 ReLU 在 $x < 0$ 时梯度恒为 0），所以梯度信号不会完全消失。

4.3 因果掩码（§ 3.3）

原文翻译

Transformer 架构可以通过掩码注意力计算来高效训练自回归模型，使得第 i 个位置只能被位置 j（$j \leq i$）影响，即一个位置不能被后续位置影响。形式上，该因果掩码将公式 3 更改如下：

$$V_i' = \frac{\sum_{j=1}^{i} \operatorname{sim}(Q_i, K_j) V_j}{\sum_{j=1}^{i} \operatorname{sim}(Q_i, K_j)}. \tag{8}$$

按照 § 4.2 的推理，我们线性化掩码注意力如下：

$$V_i' = \frac{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j)}. \tag{9}$$

通过引入 S_i 和 Z_i：

$$S_i = \sum_{j=1}^{i} \phi(K_j) V_j^T, \tag{10}$$

$$Z_i = \sum_{j=1}^{i} \phi(K_j), \tag{11}$$

我们可以将公式 9 简化为：

$$V_i' = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i}. \tag{12}$$

注意 S_i 和 Z_i 可以在常数时间内从 S_{i-1} 和 Z_{i-1} 计算：$S_i = S_{i-1} + \phi(K_i) V_i^T$，$Z_i = Z_{i-1} + \phi(K_i)$，因此带因果掩码的线性 Transformer 的计算复杂度相对于序列长度是线性的。

通俗解释

因果掩码是自回归模型（如 GPT 做文本生成、逐像素生成图片）的核心要求：生成第 i 个 token 时，只能看到前 $i-1$ 个 token，不能“偷看”后面的内容。
在标准 Transformer 中，因果掩码通过将注意力矩阵的上三角部分设为 $-\infty$（softmax 后变为 0）来实现。
在线性 Transformer 中，因果掩码的实现更加优雅：把求和上界从 N 改为 i。这意味着 S_i（“记忆矩阵”）只汇总了前 i 个位置的信息。而且 S_i 可以从 S_{i-1} 递推得到——每次只需要加上新来的一项 $\phi(K_i) V_i^T$。
这个递推关系非常关键：它意味着我们可以像 RNN 一样逐步更新状态，不需要回头看所有历史。每一步的计算量是固定的（$\mathcal{O}(CM)$），不随时间步增长。

S_i 的递推与维度分析

$S_i = \sum_{j=1}^{i} \phi(K_j) V_j^T$，其中 $\phi(K_j) \in \mathbb{R}^C$，$V_j \in \mathbb{R}^M$。
$\phi(K_j) V_j^T$ 是一个外积，结果是 $C \times M$ 矩阵。
所以 $S_i \in \mathbb{R}^{C \times M}$，是一个**固定大小**的矩阵——不随序列长度 N 增长。
递推：$S_i = S_{i-1} + \phi(K_i) V_i^T$（加一个 $C \times M$ 矩阵，代价 $\mathcal{O}(CM)$）。
计算 V_i'：$\phi(Q_i)^T S_i$ 是 $(1 \times C) \cdot (C \times M) = 1 \times M$，代价 $\mathcal{O}(CM)$。
数值例子：设 $C = D = 2, M = 2$，序列有3个位置。

$\phi(K_1) = (0.5, 1.2)^T$, $V_1 = (0.3, 0.8)^T$: $S_1 = \phi(K_1) V_1^T = \begin{pmatrix} 0.5 \\ 1.2 \end{pmatrix} (0.3, 0.8) = \begin{pmatrix} 0.15 & 0.40 \\ 0.36 & 0.96 \end{pmatrix}$

$\phi(K_2) = (0.8, 0.3)^T$, $V_2 = (0.5, 0.2)^T$: $S_2 = S_1 + \begin{pmatrix} 0.40 & 0.16 \\ 0.15 & 0.06 \end{pmatrix} = \begin{pmatrix} 0.55 & 0.56 \\ 0.51 & 1.02 \end{pmatrix}$

如果 $\phi(Q_2) = (1.0, 0.5)^T$，则：$\phi(Q_2)^T S_2 = (1.0, 0.5) \begin{pmatrix} 0.55 & 0.56 \\ 0.51 & 1.02 \end{pmatrix} = (0.805, 1.07)$

每一步只做矩阵加法和向量-矩阵乘，代价固定，不依赖已处理的序列长度。

4.3.1 梯度计算

原文翻译

公式 12 的一个简单实现（在任何深度学习框架中）需要存储所有中间值 S_i 以计算梯度。这会将内存消耗增加 $\max(D, M)$ 倍，从而阻碍因果线性注意力在更长序列或更深模型上的适用性。为了解决这个问题，我们将公式 9 分子的梯度推导为累积和。这使我们能够在**线性时间**和**常数内存**中计算因果线性注意力的前向和反向传播。详细推导在补充材料中提供。
给定分子 $\bar{V}_i$ 和标量损失函数关于分子的梯度 $\nabla_{\bar{V}_i}\mathcal{L}$，我们推导 $\nabla_{\phi(Q_i)}\mathcal{L}$、$\nabla_{\phi(K_i)}\mathcal{L}$ 和 $\nabla_{V_i}\mathcal{L}$ 如下：

$$\nabla_{\phi(Q_i)}\mathcal{L} = \nabla_{\bar{V}_i}\mathcal{L} \left(\sum_{j=1}^{i} \phi(K_j) V_j^T \right)^T, \tag{13}$$

$$\nabla_{\phi(K_i)}\mathcal{L} = \left(\sum_{j=i}^{N} \phi(Q_j) \left(\nabla_{\bar{V}_j}\mathcal{L} \right)^T \right) V_i, \tag{14}$$

$$\nabla_{V_i}\mathcal{L} = \left(\sum_{j=i}^{N} \phi(Q_j) \left(\nabla_{\bar{V}_j}\mathcal{L} \right)^T \right)^T \phi(K_i). \tag{15}$$

公式 9、13–15 中的累积和项在线性时间内计算，且关于序列长度只需常数内存。这产生了计算复杂度为 $\mathcal{O}(NCM)$、内存为 $\mathcal{O}(N \max(C, M))$ 的算法，其中 C 是特征映射的维度。分子前向和反向传播的伪代码在算法 1 中给出。

通俗解释

这是一个重要的工程细节：虽然前向传播可以用递推实现（$S_i = S_{i-1} + \ldots$），但如果直接用深度学习框架的自动微分，它会存储所有中间的 $S_1, S_2, \ldots, S_N$，内存变成 $\mathcal{O}(N \cdot C \cdot M)$。
作者手动推导了梯度公式，发现：

1. Q 的梯度包含一个“前向累积和”（从 $j = 1$ 到 i）——和前向传播方向相同；

2. K 和 V 的梯度包含一个“反向累积和”（从 $j = N$ 到 i）——和 RNN 中的“反向传播通过时间”（BPTT）方向相同。

所有这些累积和都可以用递推实现，不需要存储中间状态矩阵，因此实现了常数内存。作者用约 200 行 CUDA 代码实现了这个自定义的前向/反向传播。

梯度公式 13 的推导（补充材料内容）

设分子 $\bar{V}_{ie} = \sum_{d=1}^{D} Q_{id} \sum_{j=1}^{i} K_{jd} V_{je} = \sum_{d=1}^{D} \sum_{j=1}^{i} Q_{id} K_{jd} V_{je}$，其中下标 i 是序列位置，e 是值的维度索引，d 是特征映射维度索引（这里假设 Q, K 已经被 ϕ 映射）。
关于 Q_{lt} 求偏导：

$$\frac{\partial \mathcal{L}}{\partial Q_{lt}} = \sum_{e=1}^{M} \frac{\partial \mathcal{L}}{\partial \bar{V}_{le}} \frac{\partial \bar{V}_{le}}{\partial Q_{lt}}$$

注意 Q_{lt} 只影响 $\bar{V}_l$（因为在分子中 Q_i 只出现在位置 $i = l$ 对应的项中），所以：

$$\frac{\partial \bar{V}_{le}}{\partial Q_{lt}} = \sum_{j=1}^{l} K_{jt} V_{je}$$

因此：

$$\frac{\partial \mathcal{L}}{\partial Q_{lt}} = \sum_{e=1}^{M} \frac{\partial \mathcal{L}}{\partial \bar{V}_{le}} \sum_{j=1}^{l} K_{jt} V_{je}$$

写成向量形式：$\nabla_{Q_i} \mathcal{L} = \nabla_{\bar{V}_i} \mathcal{L} \left(\sum_{j=1}^{i} K_j V_j^T \right)^T$
其中 $\sum_{j=1}^{i} K_j V_j^T = S_i$（前向传播中已经计算了！），所以 Q 的梯度可以在前向传播的同一趟中计算。
K 和 V 的梯度需要一个反向的累积和 $\sum_{j=i}^{N} Q_j (\nabla_{\bar{V}_j} \mathcal{L})^T$，从 $j = N$ 到 i 递减求和，同样是 $\mathcal{O}(N)$ 时间和常数内存。

4.3.2 训练与推理

原文翻译

在训练自回归 Transformer 模型时，完整的真实序列是可用的。这使得层间并行化成为可能，既适用于公式 1 的 $f_l(\cdot)$，也适用于注意力计算。因此，Transformer 的训练比循环神经网络更高效。另一方面，在推理期间，时间步 i 的输出是时间步 $i+1$ 的输入。这使得自回归模型无法并行化。此外，Transformer 的每时间步代价不是常数的；相反，它随当前序列长度的平方增长，因为必须计算所有先前时间步的注意力。
我们提出的线性 Transformer 模型**结合了两者的优势**。在训练方面，计算可以并行化并充分利用 GPU 或其他加速器。在推理方面，我们的模型每一次预测的时间和内存代价是常数的。这意味着我们可以简单地将 $\phi(K_j) V_j^T$ 矩阵作为内部状态存储，并在每个时间步像循环神经网络一样更新它。这导致推理比其他 Transformer 模型**快数千倍**。

线性 Transformer 的“两全其美”

	训练	自回归推理
标准 Transformer	可并行（快）	每步 $\mathcal{O}(i)$（慢）
RNN	不可并行（慢）	每步 $\mathcal{O}(1)$（快）
线性 Transformer	**可并行（快）**	**每步 $\mathcal{O}(1)$（快）**

线性 Transformer 训练时像 Transformer（并行），推理时像 RNN（递推），两者优势兼得。

4.4 Transformer 就是 RNN（§ 3.4）

原文翻译

在文献中，Transformer 模型被认为是与循环神经网络根本不同的方法。然而，从 § 4.3 的因果掩码公式和前一节的讨论中可以明显看出，任何带因果掩码的 Transformer 层都可以被写成一个模型：给定输入，修改内部状态，然后预测输出，即一个循环神经网络（RNN）。注意，与 Universal Transformer 【Dehghani et al. 2018（Universal Transformer）：在深度方向上做循环。本文是在时间方向上做循环，两者不同】 不同，我们考虑的是关于时间的循环，而非深度。
在以下公式中，我们将公式 1 的 Transformer 层形式化为循环神经网络。所得 RNN 有两个隐藏状态：注意力记忆 s 和归一化记忆 z。我们使用下标表示循环中的时间步。

$$s_0 = 0, \tag{16}$$

$$z_0 = 0, \tag{17}$$

$$s_i = s_{i-1} + \phi(x_i W_K)(x_i W_V)^T, \tag{18}$$

$$z_i = z_{i-1} + \phi(x_i W_K), \tag{19}$$

$$y_i = f_l\left(\frac{\phi(x_i W_Q)^T s_i}{\phi(x_i W_Q)^T z_i} + x_i\right). \tag{20}$$

在上述公式中，x_i 表示特定 Transformer 层的第 i 个输入，y_i 表示第 i 个输出。注意我们的表述不对特征函数施加任何约束，理论上可以用于表示**任何** Transformer 模型，甚至使用 softmax 注意力的模型。这一表述是更好地理解 Transformer 与流行的循环网络 【Hochreiter & Schmidhuber 1997（LSTM）：最经典的 RNN 变体】 之间关系以及信息存储和检索过程的第一步。

通俗解释

这是论文最深刻的洞察：**每个带因果掩码的 Transformer 层，本质上就是一个 RNN！**
具体来说，把 Transformer 层改写成 RNN 形式后：

1. **隐藏状态** s_i（$C \times M$ 矩阵）：相当于“记忆矩阵”，累积了所有已看到的键-值对信息。类似 LSTM 中的细胞状态。
2. **隐藏状态** z_i（C 维向量）：归一化因子，累积了所有已看到的键的特征映射。
3. **状态更新**：$s_i = s_{i-1} + \phi(K_i) V_i^T$——简单的加法更新，就像往记忆中“写入”新信息。
4. **输出计算**：$\phi(Q_i)^T s_i / (\phi(Q_i)^T z_i)$——用查询去“读取”记忆中的信息。

与 LSTM 的类比：LSTM 通过遗忘门控制“忘记什么”，通过输入门控制“记住什么”。线性 Transformer 的 RNN 形式没有遗忘门（只有加法累积），所以理论上它会“记住一切”。这既是优点（不会遗忘）也是潜在的缺点（无法主动忘记无关信息）。

RNN 形式的维度与计算分析

维度：

- $x_i \in \mathbb{R}^F$（输入特征）
- $x_i W_K \in \mathbb{R}^D$，$\phi(x_i W_K) \in \mathbb{R}^C$（$C = D$ for elu+1）
- $x_i W_V \in \mathbb{R}^M$
- $s_i = s_{i-1} + \phi(x_i W_K)(x_i W_V)^T \in \mathbb{R}^{C \times M}$
- $z_i = z_{i-1} + \phi(x_i W_K) \in \mathbb{R}^C$
- $\phi(x_i W_Q)^T s_i \in \mathbb{R}^M$（查询结果）
- $\phi(x_i W_Q)^T z_i \in \mathbb{R}$（归一化因子，标量）

每时间步的计算量：

- 投影：$x_i W_Q, x_i W_K, x_i W_V$ 各 $\mathcal{O}(FD)$ 或 $\mathcal{O}(FM)$
- 特征映射：$\mathcal{O}(D)$（逐元素 elu+1）
- 外积：$\phi(K_i) V_i^T$ 需要 $\mathcal{O}(CM)$
- 状态更新：$\mathcal{O}(CM)$（矩阵加法）
- 查询：$\phi(Q_i)^T s_i$ 需要 $\mathcal{O}(CM)$

总计：每步 $\mathcal{O}(F(D+M)+CM)$，完全不依赖 i 或 N！
对比标准 Transformer 推理：第 i 步需要计算 Q_i 与所有 $K_1, \ldots, K_i$ 的注意力，代价 $\mathcal{O}(iD+iM)$。总推理代价 $\sum_{i=1}^{N} \mathcal{O}(iD) = \mathcal{O}\left(N^2 D\right)$。

5 实验 (Experiments)

5.1 算法：线性 Transformer 因果掩码的前向与反向传播

Algorithm 1 线性 Transformer 因果掩码（Linear Transformers with Causal Masking）

Function Forward($\phi(Q), \phi(K), V$):
1: $\bar{V} \leftarrow 0, S \leftarrow 0$
2: **for** $i = 1, \ldots, N$ **do**
3: $\quad S \leftarrow S + \phi(K_i) V_i^T$ ▷ 公式 10：更新记忆矩阵
4: $\quad \bar{V}_i \leftarrow \phi(Q_i) S$ ▷ 用查询读取记忆
5: **end for**
6: **return** $\bar{V}$
7:
Function Backward($\phi(Q), \phi(K), V, G$):
8: // G 是损失关于 Forward 输出的梯度
9: $S \leftarrow 0, \nabla_{\phi(Q)}\mathcal{L} \leftarrow 0$
10: **for** $i = 1, \ldots, N$ **do**
11: $\quad S \leftarrow S + \phi(K_i) V_i^T$ ▷ 前向累积和
12: $\quad \nabla_{\phi(Q_i)}\mathcal{L} \leftarrow G_i S^T$ ▷ 公式 13
13: **end for**
14: $S \leftarrow 0, \nabla_{\phi(K)}\mathcal{L} \leftarrow 0, \nabla_V \mathcal{L} \leftarrow 0$
15: **for** $i = N, \ldots, 1$ **do**
16: $\quad S \leftarrow S + \phi(Q_i) G_i^T$ ▷ 反向累积和
17: $\quad \nabla_{V_i}\mathcal{L} \leftarrow S^T \phi(K_i)$ ▷ 公式 15
18: $\quad \nabla_{\phi(K_i)}\mathcal{L} \leftarrow S V_i$ ▷ 公式 14
19: **end for**
20: **return** $\nabla_{\phi(Q)}\mathcal{L}, \nabla_{\phi(K)}\mathcal{L}, \nabla_V \mathcal{L}$

算法 1 逐行解读

前向传播（Forward）：

- **第1行**：初始化输出 $\bar{V} = 0$ 和记忆矩阵 $S = 0$（$S \in \mathbb{R}^{C \times M}$）。
- **第2行**：遍历序列的每个位置 i。
- **第3行**：更新 S——将当前位置的键-值外积加入记忆。$\phi(K_i) \in \mathbb{R}^C$, $V_i \in \mathbb{R}^M$，外积 $\phi(K_i) V_i^T \in \mathbb{R}^{C \times M}$。这是“写入”操作。设计动机：$S_i$ 只需从 S_{i-1} 加上一项即可，不需要重新计算。
- **第4行**：用查询 $\phi(Q_i)$ 去“读取”记忆 S，得到分子 $\bar{V}_i \in \mathbb{R}^M$。
- **第5行**：返回所有位置的分子值。

反向传播（Backward）：

- **第8–12行**：计算 Q 的梯度。用**前向方向**（$i = 1 \to N$）的累积和。S 在这里和前向传播的意义相同（累积 $\phi(K_j) V_j^T$），所以可以复用前向传播的 S。
- **第13–18行**：计算 K 和 V 的梯度。用**反向方向**（$i = N \to 1$）的累积和。这里累积的是 $\phi(Q_j) G_j^T$（查询和梯度的外积），类似于 RNN 中的“反向传播通过时间”（BPTT）。

内存分析：整个算法只需维护一个 $C \times M$ 的累积矩阵 S——常数内存！不需要存储 $S_1, S_2, \ldots, S_N$。

原文翻译

在本节中，我们实验分析了所提出的线性 Transformer 的性能。首先，在 § 4.1 中，我们在合成数据上评估线性化注意力的计算代价、内存消耗和收敛性。为了进一步展示线性 Transformer 的有效性，我们在两个实际应用上评估了我们的模型：§ 4.2 的图像生成和 § 4.3 的自动语音识别。我们表明我们的模型相对于最先进的 Transformer 架构实现了有竞争力的性能，同时需要显著更少的 GPU 内存和计算。
在所有实验中，我们将我们的模型与两个基线进行比较：具有 softmax 注意力的完整 Transformer，以及 Reformer 【Kitaev et al. 2020：基于 LSH 的高效 Transformer，是当时最先进的加速方法】，后者是最先进的加速 Transformer 架构。对于 Reformer，我们使用已发布代码的 PyTorch 重新实现；对于完整 Transformer，我们使用默认的 PyTorch 实现。注意对于 Reformer，我们不使用可逆层，但这不影响结果，因为我们仅测量自注意力层的内存消耗。在所有实验中，我们使用 **softmax** 指代标准 Transformer 架构，**linear** 指代我们提出的线性 Transformer，**lsh-X** 指代 Reformer，其中 X 表示哈希轮数。
对于训练线性 Transformer，我们使用公式 7 的特征映射。我们的 PyTorch 代码及文档和示例可以在 `https://linear-transformers.com/` 找到。公式 13–15 的常数内存梯度计算用大约 200 行 CUDA 代码实现。

5.2 合成任务（§ 4.1）

5.2.1 收敛性分析

原文翻译

为了检验线性 Transformer 的收敛特性，我们在一个带因果掩码的人工复制任务上训练。即 Transformer 必须复制一系列符号，类似于 Kitaev 等人（2020）的序列复制任务。我们使用最大长度为 128 的序列，包含 10 个不同的符号，由专用分隔符分隔。对于所有三种方法，我们训练一个 4 层 Transformer，每层 8 个注意力头，使用批大小 64 和 RAdam 优化器 【Liu et al. 2019：提出 RAdam——一种带学习率预热的 Adam 变体】，学习率为 10^{-3}，在 3000 次更新后降至 10^{-4}。图 1 展示了损失关于梯度步数的变化。我们观察到 linear 收敛平滑，由于缺少哈希引入的噪声，达到了比 lsh 更低的损失。特别是，它达到了与 softmax 相同的损失。

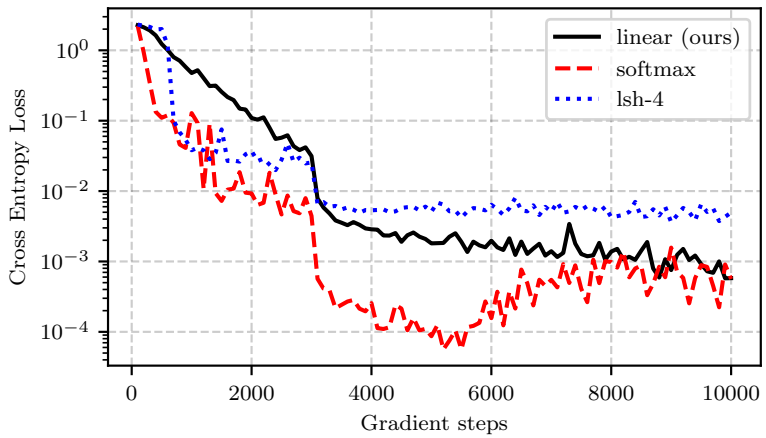


Figure 1: 在序列复制任务上 softmax、linear 和 Reformer 注意力的收敛比较。linear 收敛稳定，达到了与 softmax 相同的最终性能。

图 1 详细解读

图形内容：横轴是梯度更新步数，纵轴是训练损失（越低越好）。共有三条曲线：softmax（标准 Transformer）、linear（本文方法）和 lsh（Reformer）。
关键观察：

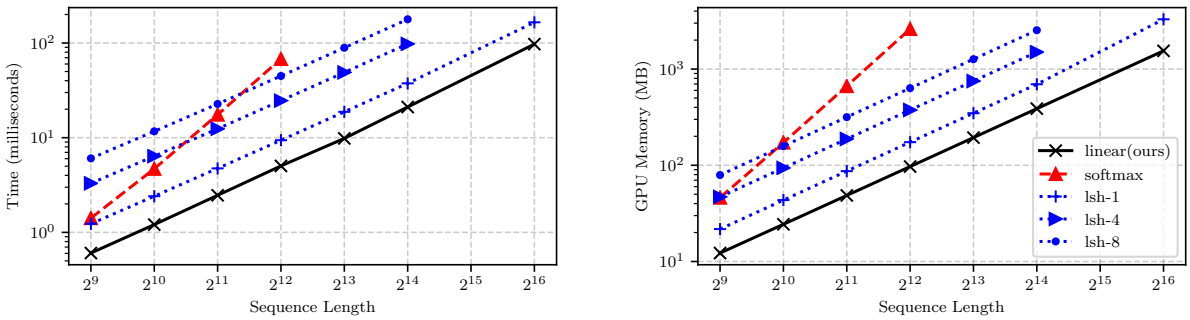
1. **linear 的收敛曲线平滑**：因为线性注意力是确定性的，没有引入额外噪声。
2. **lsh 的曲线有较大波动**：因为 LSH 哈希是随机的，引入了噪声，尤其是只用 1 轮哈希时。
3. **linear 最终达到了与 softmax 相同的损失**：说明在这个任务上，线性注意力没有性能损失。
4. **lsh 的最终损失略高**：哈希近似引入的误差导致性能略有下降。

这张图支持的论点：线性注意力不仅速度快，而且收敛质量与 softmax 注意力相当。

5.2.2 内存和计算需求

原文翻译

在本小节中，我们比较 Transformer 的计算和内存需求。我们对序列长度 $N \in \{2^9, 2^{10}, \ldots, 2^{16}\}$ 不同的合成输入计算注意力及其梯度，并测量每种 Transformer 变体的 GPU 峰值分配内存和所需时间。我们将批大小与序列长度成反比地缩放，并报告批中每个样本的时间和内存。
每种方法评估到能放入 GPU 内存的最大序列长度。对于此基准测试，我们使用一块 11GB 内存的 NVidia GTX 1080 Ti。结果是 softmax 的最大序列长度为 4,096 个元素，lsh-4 和 lsh-8 为 16,384。如预期，softmax 相对于序列长度呈二次增长。我们的方法在每种配置下都比基线更快且内存更少，如图 2 所示。我们观察到 Reformer 和线性注意力都相对于序列长度线性增长。注意虽然 Reformer 的渐近复杂度是 $\mathcal{O}(N \log N)$，但 $\log N$ 足够小，不影响实际计算时间。



(a) 计算时间　　(b) 内存消耗

Figure 2: Reformer (lsh-X)、softmax 注意力和 linear 注意力的前向/反向传播计算需求比较。Linear 和 Reformer 模型相对于序列长度线性增长，而 softmax 在内存和时间上都呈二次增长。

图 2 详细解读

左图（计算时间）： 横轴是序列长度（2^9 到 2^{16}，即 512 到 65536），纵轴是每个样本的计算时间。softmax 的曲线呈明显的二次增长（抛物线形状），在 $N = 4096$ 时已经耗尽内存无法继续。linear 和 lsh 的曲线近似线性增长，而 linear 始终低于 lsh——在所有序列长度下都是最快的。
右图（内存消耗）： 同样的横轴，纵轴是 GPU 峰值内存。softmax 的内存呈二次增长（因为要存 $N \times N$ 的注意力矩阵），而 linear 的内存增长最缓慢。
关键数字： 在序列长度 $2^{16} = 65536$ 时，softmax 已经完全无法运行，而 linear 仍然可以正常工作——这就是线性复杂度的实际优势。

5.3 图像生成（§ 4.2）

原文翻译

Transformer 在条件或无条件自回归生成任务上展示了出色的结果【Radford et al. 2019（GPT-2）；Child et al. 2019（Sparse Transformer）：展示了 Transformer 在生成任务上的成功】，然而，由于任务本身是顺序的且内存随序列长度平方增长，从 Transformer 中采样很慢。在本节中，我们训练因果掩码 Transformer 逐像素预测图像。我们在每维比特数方面的性能与 softmax 注意力持平，同时能够**快 1,000 倍以上**生成图像，且从第一个到最后一个像素**内存恒定**。我们在补充材料中提供了训练演化、生成图像质量和生成单张图像时间的比较。此外，我们还与一个在推理时缓存键和值的更快的 softmax Transformer 进行了比较（与 PyTorch 默认实现相对）。

5.3.1 MNIST（§ 4.2.1）

原文翻译

首先，我们在广泛使用的 MNIST 数据集【LeCun et al. 2010：手写数字数据集，图像大小 28×28 = 784 像素】上用自回归 Transformer 评估图像生成。该实验的架构包含 8 个注意力层，每层 8 个注意力头。我们将嵌入大小设为 256，即每个头 32 维。前馈维度是嵌入大小的 4 倍。我们用 10 个 logistic 混合来建模输出【Salimans et al. 2017（PixelCNN++）：提出用离散化 logistic 混合建模像素值分布】。我们使用 RAdam 优化器，学习率为 10^{-4}，所有模型训练 250 个 epoch。对于 Reformer 基线，我们使用 1 和 4 轮哈希。此外，按照 Kitaev 等人（2020）的建议，我们使用 64 个桶和约 32 个元素的块。特别是，我们将 783 长的输入序列分成 27 个块，每块 29 个元素。由于序列长度相对较小（仅 784 个像素），为消除不同批大小的影响，我们对所有方法使用批大小 10。
表 1 总结了结果。我们观察到线性 Transformer 在最终困惑度方面达到了几乎与 softmax Transformer 相同的性能，同时生成图像的速度快 300 倍以上。这得益于我们模型的低内存需求，它能够用单个 GPU 同时生成 10,000 张 MNIST 图像。特别是，内存关于序列长度是恒定的，因为在像素之间唯一需要存储的是公式 18 和 19 中描述的 s_i 和 z_i 值。另一方面，softmax 和 Reformer 都需要随序列长度增加的内存。
我们 MNIST 模型的图像补全和无条件采样可以在图 3 中看到。我们观察到线性 Transformer 生成了非常令人信服的样本，具有清晰的边界且无噪声。在图像补全的情况下，我们还观察到 Transformer 学会了使用与原始图像相同的笔画风格和宽度，有效地注意了长时间距离的信息。注意，由于所有模型的困惑度大致相同，我们没有观察到不同模型生成样本之间的质性差异。

方法	Bits/dim	Images/sec	
Softmax	0.621	0.45	(1×)
LSH-1	0.745	0.68	(1.5×)
LSH-4	0.676	0.27	(0.6×)
Linear (ours)	0.644	**142.8**	**(317×)**

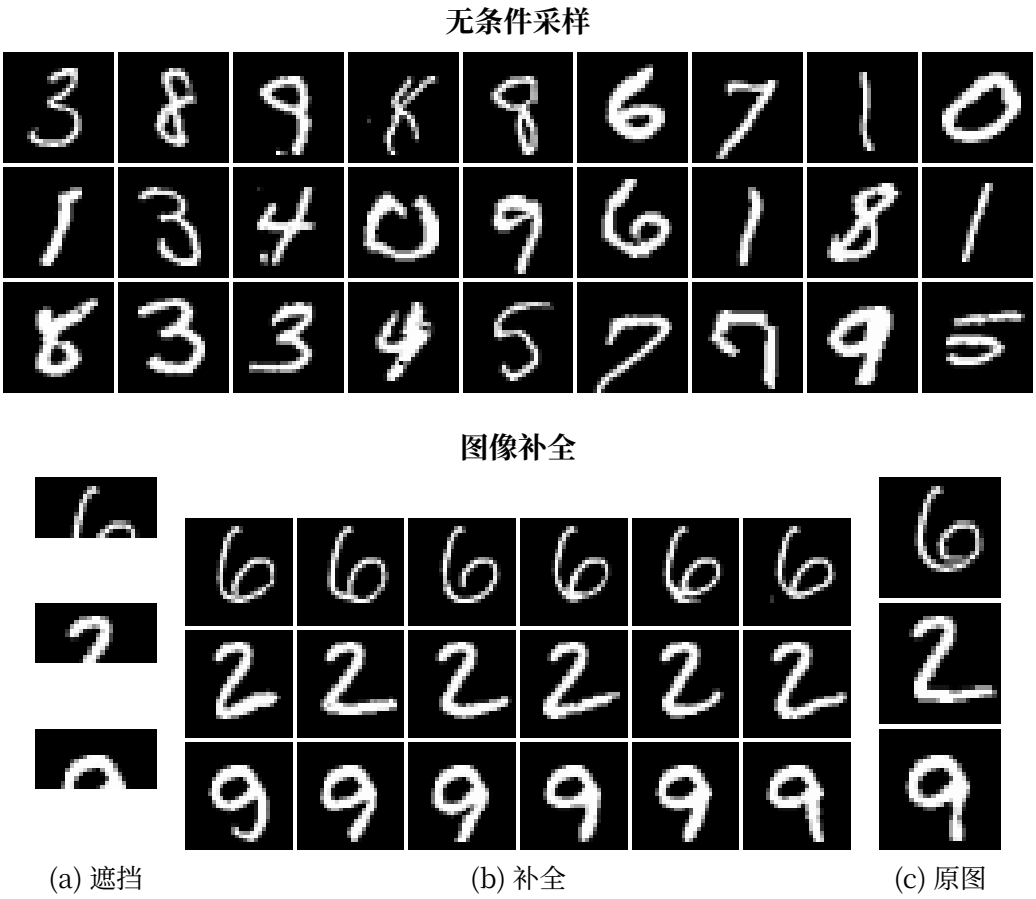
Table 1: MNIST 图像自回归生成比较。线性 Transformer 达到了几乎与 softmax 相同的 bits/dim，但图像生成吞吐量高出 300 倍以上。

表 1 解读

Bits/dim：每维比特数，衡量模型的建模质量，越低越好。Softmax 最优（0.621），Linear 紧随其后（0.644），差距仅 0.023——几乎可以忽略。LSH-1 最差（0.745），说明只用 1 轮哈希近似太粗糙。

Images/sec：每秒生成的图像数，衡量推理速度。Linear 达到 142.8 张/秒，是 softmax（0.45 张/秒）的 **317 倍**！这就是 RNN 式推理的威力——每步只需更新固定大小的状态矩阵，可以同时在 GPU 上并行生成大量图像。

为什么 LSH-4 反而比 LSH-1 慢？ 4 轮哈希虽然更精确（bits/dim 更低），但计算量也更大（每次注意力计算要做 4 次哈希和排序）。



(a) 遮挡 (b) 补全 (c) 原图

Figure 3: 我们方法在 MNIST 上的无条件采样和图像补全。(a) 展示遮挡的原始图像，(b) 补全结果，(c) 原始图像。模型达到与 softmax 可比的 bits/dim，同时吞吐量高出 300 倍以上，每秒生成 142 张图像。

图 3 详细解读

上半部分（无条件采样）：模型从零开始逐像素生成手写数字图像。可以看到生成的数字清晰、多样，笔画自然。
下半部分（图像补全）：(a) 遮挡图：原始图像的下半部分被遮挡（白色区域）；(b) 补全结果：模型根据上半部分信息，自回归地生成下半部分——注意它保持了笔画的风格和宽度；(c) 原图：用于参考。
这张图支持的论点：线性 Transformer 生成的图像质量与 softmax Transformer 无明显差异，验证了“速度提升不以质量为代价”的主张。

5.3.2 CIFAR-10 (§ 4.2.2)

原文翻译

我们的线性表述的优势随序列长度增加而增大。为了展示这一点，我们训练 16 层 Transformer 生成 CIFAR-10 图像【Krizhevsky et al. 2009：CIFAR-10 数据集，32×32×3 = 3072 子像素】。每层使用与前一实验相同的配置。对于 Reformer，我们同样使用 64 个桶和 83 个 37 元素的块（约为推荐的 32）。由于序列长度几乎是前一实验的 4 倍，完整 Transformer 在我们能用的最大 GPU（24GB 内存的 NVidia P40）上只能使用批大小 1。对于线性 Transformer 和 Reformer，我们使用批大小 4。所有模型训练 7 天。我们在表 2 中报告每维比特数和图像生成吞吐量。注意虽然该实验的主要关注点不是最终困惑度，但很明显，随着序列长度的增长，快速 Transformer 模型每 GPU 小时变得越来越高效，达到了比较慢的对应模型更好的分数。
由于生成单个像素的内存和时间对 Reformer 和 softmax 注意力都呈二次增长，线性 Transformer 吞吐量的提升更加显著。特别是，**softmax Transformer 每生成一张图像，我们的方法可以生成 4,460 张**。我们模型的图像补全和无条件采样见图 4。我们观察到模型生成了具有空间一致性的图像，并能令人信服地补全图像，而不会显著损害图像类别的识别。例如，在图 4b 中，所有图像都成功补全了狗的鼻子（第一行）或卡车的挡风玻璃（最后一行）。

方法	Bits/dim	Images/sec	
Softmax	3.47	0.004	(1×)
LSH-1	3.39	0.015	(3.75×)
LSH-4	3.51	0.005	(1.25×)
Linear (ours)	3.40	**17.85**	**(4,462×)**

Table 2: 在单个 GPU 上训练 1 周的 CIFAR-10 自回归 Transformer。线性 Transformer 完成的 epoch 数是 softmax 的 3 倍，因此困惑度更好。生成速度快 4,000 倍以上。

表 2 解读

CIFAR-10 的序列长度（$32 \times 32 \times 3 = 3072$）是 MNIST（$28 \times 28 = 784$）的约 4 倍。二次复杂度在这里的惩罚更大：
Bits/dim：Linear（3.40）甚至比 Softmax（3.47）更好！这不是因为 Linear 模型更强，而是因为在固定 7 天训练时间内，Linear 由于效率更高，完成了更多 epoch。LSH-1 最好（3.39），因为它训练 epoch 也多且没有太多近似误差。
Images/sec：Softmax 每秒只能生成 0.004 张图像（约 4 分钟一张），而 Linear 达到 17.85 张/秒——**4,462 倍**的加速！从 MNIST 的 317 倍加速到 CIFAR-10 的 4,462 倍，清

晰展示了随着序列增长，线性复杂度的优势呈指数级放大。

(a) 遮挡 (b) 补全 (c) 原图

Figure 4: 我们方法在 CIFAR-10 上的无条件采样和图像补全。随着序列长度增长，线性 Transformer 相比 softmax 注意力更加高效。吞吐量高出 4,000 倍以上，每秒生成 17.85 张图像。

图 4 详细解读

无条件采样（上半部分）：从零开始生成的 CIFAR-10 图像，可以看到图像虽然分辨率低（32×32），但具有空间一致性——颜色分布合理，可以辨认出物体轮廓。
图像补全（下半部分）： (a) 遮挡图：上半部分可见，下半部分被遮挡；(b) 补全结果：模型根据可见部分预测遮挡区域。如作者所说，狗的鼻子和卡车的挡风玻璃都被合理地补全；(c) 原图作为参照。
补全质量：虽然不完美，但考虑到只有 32×32 分辨率且只看到了图像的上半部分，这个结果相当不错。

5.4 自动语音识别（§ 4.3）

原文翻译

为了展示我们的方法也可以用于非自回归任务，我们在端到端自动语音识别中评估了线性 Transformer 的性能，使用连接时序分类（CTC）损失【Graves et al. 2006：提出 CTC 损失，用于序列到序列的对齐问题】。在该设置中，我们以非自回归方式预测每个

输入帧的音素分布。我们使用 80 小时的 WSJ 数据集 【Paul & Baker 1992：华尔街日报语音数据集】，以 40 维梅尔频谱滤波器组（不含时间差分）作为特征。数据集包含平均 800 帧、最长 2,400 帧的序列。
对于此任务，我们还与双向 LSTM 【Hochreiter & Schmidhuber 1997（LSTM）：经典循环网络】进行比较，使用 3 层隐藏大小 320。我们使用 Adam 优化器 【Kingma & Ba 2014：自适应学习率优化器】，学习率为 10^{-3}，当验证误差停止下降时降低。对于 Transformer 模型，我们使用 9 层 6 头，嵌入维度与图像实验相同。优化器使用 RAdam，初始学习率 10^{-4}，验证误差停止下降时除以 2。
所有模型以音素错误率（PER）和每 epoch 训练时间评估。我们观察到 linear 在性能和速度上都以大幅度优于循环网络基线和 Reformer，如表 3 所示。注意 softmax Transformer 在所有基线中达到了最低的音素错误率，但速度显著更慢。特别是，线性 Transformer 每 epoch 快 3 倍以上。我们在补充材料中提供了训练演化图。

方法	验证 PER	每 epoch 时间 (秒)
Bi-LSTM	10.94	1047
Softmax	5.12	2711
LSH-4	9.33	2250
Linear (ours)	8.08	**824**

Table 3: WSJ 数据集上自动语音识别性能比较。结果以音素错误率（PER）和每 epoch 训练时间给出。线性 Transformer 优于 LSTM 和 Reformer，且训练更快。

表 3 解读

这是一个**非自回归**任务（所有输出位置可以并行预测），所以这里测试的不是推理加速，而是**训练效率**和**建模质量**。
PER（音素错误率，越低越好）：Softmax 最优（5.12），但 Linear（8.08）远好于 Bi-LSTM（10.94）和 LSH-4（9.33）。
每 epoch 时间：Linear 最快（824秒），是 Softmax（2711秒）的约 1/3。这意味着在相同时间内 Linear 可以训练更多 epoch。
Softmax 为什么 PER 更好? 语音识别中自注意力的“全局视角”（每帧看到所有其他帧）对于建模长距离语音依赖非常重要。线性近似丢失了一些精度。但考虑到 3 倍速度提升，这是一个合理的权衡。
Reformer 为什么最差? LSH 的键=查询约束在这个任务中限制了模型的表达能力。

6 结论 (Conclusions)

原文翻译

在这项工作中，我们提出了线性 Transformer，一种显著减少原始 Transformer 的内存和计算成本的模型。特别是，通过利用矩阵乘积的结合律，我们能够在时间和内存上以相对于序列长度线性增长的方式计算自注意力。我们表明我们的模型可以与因果掩码一起使用，并且仍然保留其线性渐近复杂度。最后，我们将 Transformer 模型表示为循环神经网络，这使我们能够在自回归任务上以快数千倍的速度执行推理。
这一特性为关于 RNN 和 Transformer 中信息存储和检索的未来研究打开了多种方向。另一条待探索的研究方向与线性注意力的特征映射选择有关。例如，用随机傅里叶特征近似 RBF 核可以使我们使用用 softmax 注意力预训练的模型。

通俗解释

结论总结了三大贡献，并提出了两个未来方向：
方向一： RNN 和 Transformer 信息存储机制的统一理解——既然它们可以相互转化，那它们的记忆机制有什么异同？
方向二： 更好的特征映射——如果能找到一个有限维的 ϕ 很好地近似 softmax 的指数核（比如用随机傅里叶特征），那就可以把已有的 softmax 预训练模型直接转换为线性模型，免去重新训练的代价。这个方向后来被多项后续工作探索（如 Performer 等）。

7 致谢 (Acknowledgements)

原文翻译

Angelos Katharopoulos 由瑞士国家科学基金资助，资助编号 FNS-30209 “ISUL” 和 FNS-30224 “CORTI”。Apoorv Vyas 由瑞士国家科学基金资助，资助编号 FNS-30213 “SHISSM”。Nikolaos Pappas 由瑞士国家科学基金资助，资助编号 P400P2_183911 “UNISON”。

A 附录 A：梯度推导 (Gradient Derivation)

原文翻译

在补充材料的第一节中，我们详细推导了因果掩码线性 Transformer 的梯度，并展示它们可以在线性时间和常数内存中计算。特别地，我们推导标量损失关于以下公式分子的梯度：

$$V_i' = \frac{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j)}. \tag{21}$$

分母和分式的梯度由 autograd 高效处理。不失一般性，我们可以假设 Q 和 K 已经包含了被 $\phi(\cdot)$ 映射后的向量，因此给定分子：

$$\bar{V}_i = Q_i^T \sum_{j=1}^{i} K_j V_j^T, \tag{22}$$

和 $\nabla_{\bar{V}}\mathcal{L}$，我们寻求计算 $\nabla_Q\mathcal{L}$、$\nabla_K\mathcal{L}$ 和 $\nabla_V\mathcal{L}$。注意 $Q \in \mathbb{R}^{N\times D}$，$K \in \mathbb{R}^{N\times D}$，$V \in$

$\mathbb{R}^{N\times M}$。为推导梯度，我们首先不用向量记号表示上式的单个元素：

$$\bar{V}_{ie} = \sum_{d=1}^{D} Q_{id} \sum_{j=1}^{i} K_{jd} V_{je} = \sum_{d=1}^{D} \sum_{j=1}^{i} Q_{id} K_{jd} V_{je}. \tag{23}$$

随后，我们通过对任意 Q_{lt} 求偏导来推导 Q 的梯度：

$$\frac{\partial \mathcal{L}}{\partial Q_{lt}} = \sum_{e=1}^{M} \frac{\partial \mathcal{L}}{\partial \bar{V}_{le}} \frac{\partial \bar{V}_{le}}{\partial Q_{lt}} = \sum_{e=1}^{M} \frac{\partial \mathcal{L}}{\partial \bar{V}_{le}} \left(\sum_{j=1}^{l} K_{jt} V_{je} \right). \tag{24}$$

写成矩阵乘积的梯度形式：

$$\nabla_{Q_i} \mathcal{L} = \nabla_{\bar{V}_i} \mathcal{L} \left(\sum_{j=1}^{i} K_j V_j^T \right)^T, \tag{25}$$

证明了主文的公式 13。在公式 24 中，我们利用了 Q_{lt} 只影响 $\bar{V}_l$，因此不需要对 i 求和来计算梯度。
然而对于 K 和 V，情况不同。特别是 K_j 影响所有满足 $i \geq j$ 的 $\bar{V}_i$。因此，损失关于 K_{lt} 的偏导数可以写为：

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial K_{lt}} &= \sum_{e=1}^{M} \sum_{i=l}^{N} \frac{\partial \mathcal{L}}{\partial \bar{V}_{ie}} \frac{\partial \bar{V}_{ie}}{\partial K_{lt}} = \sum_{e=1}^{M} \sum_{i=l}^{N} \frac{\partial \mathcal{L}}{\partial \bar{V}_{ie}} \frac{\partial \left(\sum_{d=1}^{D} \sum_{j=1}^{i} Q_{id} K_{jd} V_{je} \right)}{\partial K_{lt}} \\ &= \sum_{e=1}^{M} \sum_{i=l}^{N} \frac{\partial \mathcal{L}}{\partial \bar{V}_{ie}} Q_{it} V_{le}. \end{aligned} \tag{26}$$

和对 Q 一样，我们现在可以写成向量化形式：

$$\nabla_{K_i} \mathcal{L} = \left(\sum_{j=i}^{N} Q_j \left(\nabla_{\bar{V}_j} \mathcal{L} \right)^T \right) V_i, \tag{27}$$

证明了主文的公式 14。按照同样的推理，可以计算损失关于 V_{lt} 的偏导数，从而证明公式 15。
注意 Q 和 K 梯度的累积和矩阵大小相同，但一个在前向方向计算（从 1 到 N 求和），类似于前向传播，另一个在反向方向计算（从 N 到 1 求和），类似于 RNN 中的反向传播通过时间。

K 梯度的推导细节

为什么 K 的梯度需要反向累积和?
关键在于 K_j 出现在所有 $\bar{V}_i$（$i \geq j$）中。这是因为因果掩码的求和上界是 i，所以 K_j 在所有 $i \geq j$ 的位置都有贡献。
展开来看（以位置 l 和维度 t 为例）：

$$\frac{\partial \mathcal{L}}{\partial K_{lt}} = \sum_{e=1}^{M} \sum_{i=l}^{N} \frac{\partial \mathcal{L}}{\partial \bar{V}_{ie}} Q_{it} V_{le}$$

注意求和是从 $i = l$ 到 N（不是从 1 到 l！）。向量化后：

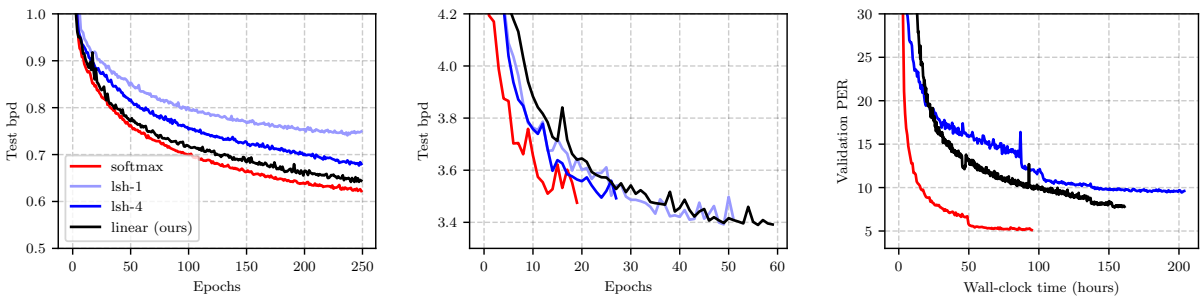
$$\nabla_{K_i}\mathcal{L} = \underbrace{\left(\sum_{j=i}^{N} Q_j(\nabla_{\bar{V}_j}\mathcal{L})^T\right)}_{\text{从 } N \text{ 到 } i \text{ 的反向累积和}} V_i$$

这个反向累积和可以用如下递推计算：设 $R_N = Q_N(\nabla_{\bar{V}_N}\mathcal{L})^T$，然后 $R_i = R_{i+1} + Q_i(\nabla_{\bar{V}_i}\mathcal{L})^T$。
每步只需做一次外积加法（$\mathcal{O}(DM)$），总计 $\mathcal{O}(NDM)$，内存常数。

B 附录 B：训练演化 (Training Evolution)

原文翻译

在图 5 中，我们展示了所有实验中所有 Transformer 模型的训练演化。对于 MNIST 实验（图 5a），我们对所有方法训练 250 个 epoch。序列长度足够小，所有方法的训练时间没有显著差异。我们观察到我们的方法收敛速度与 softmax 注意力持平，显著优于两个 Reformer 变体。
另一方面，对于 CIFAR-10（图 5b），我们对所有方法训练固定时间，即 7 天。我们观察到 lsh-1 和 linear 完成的 epoch 数显著多于 softmax 和 lsh-4，并达到了更好的性能。随着序列长度的进一步增加，这一差距预计会加大。
最后，在我们最后一个自动语音识别实验中（图 5c），softmax 在收敛方面显著优于 Reformer 和 linear。注意 linear 每 epoch 快 3 倍，这意味着在与 softmax 相同的时间内它已经完成了大约 4 倍的 epoch。虽然 softmax 注意力在此任务中更好，但我们观察到线性 Transformer 在收敛和最终性能方面都显著优于 Reformer。



(a) MNIST　(b) CIFAR-10　(c) 语音识别

Figure 5: 所有实验的 Transformer 训练演化。可以观察到线性 Transformer 始终比 Reformer 收敛更快，在自回归实验中与 softmax 持平。MNIST 训练 250 epoch，CIFAR-10 训练 7 天，语音识别训练到收敛。

图 5 详细解读

(a) MNIST：横轴是 epoch 数，纵轴是 bits/dim。所有方法训练 250 个 epoch。Linear 和 softmax 收敛曲线几乎重合，最终达到相似的 bits/dim。LSH 变体收敛较慢且最终性能较差。
(b) CIFAR-10：横轴是 epoch 数，但注意所有方法训练的是固定 7 天。Linear 和 LSH-1 在 7 天内完成了更多 epoch（因为每 epoch 更快），所以曲线延伸更远。这说明在固

定时间预算下，效率更高的模型可以训练更充分。
(c) 语音识别：Softmax 收敛曲线明显低于其他方法（PER 更低=更好）。但 Linear 明显优于 LSH-4。Softmax 在这个任务上的优势说明全 softmax 注意力对语音识别更重要，线性近似带来了一定性能损失。
整体趋势：在自回归任务（MNIST、CIFAR-10）中，Linear 和 Softmax 性能相当；在非自回归任务（语音识别）中，Softmax 更优但 Linear 仍显著好于 Reformer。

C 附录 C：图像生成吞吐量讨论

C.1 有状态 softmax 注意力 (Stateful Softmax Attention)

原文翻译

在主文 § 4.2 中，我们报告了图像生成吞吐量，并与 softmax Transformer 和 lsh 进行了比较。在本节中，我们创建了另一个基线，记为 **stateful-softmax**，它将 softmax 自回归 Transformer 实现为循环模型。即所有键和值都被保存，然后在预测序列的下一个元素时再次传递给模型。该循环模型的状态是键和值的集合，其大小与序列长度成正比。这与我们提出的具有固定维度状态的模型有本质区别，而且计算第 i 个状态在给定前一个状态时具有固定的计算代价，不依赖于 i。

方法	Bits/dim	Images/sec		方法	Bits/dim	Images/sec	
Softmax	0.621	0.45	(1×)	Softmax	3.47	0.004	(1×)
Stateful-softmax	0.621	7.56	(16.8×)	Stateful-softmax	3.47	0.32	(80×)
LSH-1	0.745	0.68	(1.5×)	LSH-1	3.39	0.015	(3.75×)
LSH-4	0.676	0.27	(0.6×)	LSH-4	3.51	0.005	(1.25×)
Linear (ours)	0.644	**142.8**	**(317×)**	Linear (ours)	3.40	**17.85**	**(4,462×)**

(a) MNIST 图像生成　　(b) CIFAR-10 图像生成

Table 4: MNIST 和 CIFAR-10 图像自回归生成吞吐量比较。对于 stateful-softmax，我们保存键和值并在预测下一个元素时重用。

表 4 解读

Stateful-softmax 是一个更公平的基线——它缓存了之前的 K 和 V，不需要每步重新计算所有注意力。这在实际工程中是常见的优化（如 KV-Cache）。
MNIST：Stateful-softmax 达到 7.56 imgs/sec（原版 softmax 的 16.8 倍），但 Linear 仍然是它的 **19 倍**（142.8 vs 7.56）。
CIFAR-10：差距更大。Stateful-softmax 达到 0.32 imgs/sec（80 倍于原版），但 Linear 仍然快 **56 倍**（17.85 vs 0.32）。
根本原因：Stateful-softmax 虽然不需要重新投影 K 和 V，但每步仍需计算当前 Q 与所有已缓存的 K 的注意力，代价随时间步 i 线性增长。总推理代价仍然是 $\mathcal{O}(N^2)$。而 Linear 每步代价恒定，总代价 $\mathcal{O}(N)$。

原文翻译

表 4 总结了结果。我们观察到 stateful-softmax 显著快于标准 Transformer。然而，其复杂度相对于序列长度仍然是二次的，我们的表述在 CIFAR-10 上快了 50 倍以上。此

外，我们想指出，为 Reformer 实现类似的有状态注意力并非易事，因为排序和分块操作每次收到新输入时都需要执行。

C.2 等化批大小 (Equalizing the Batch Size)

原文翻译

在前面的章节中，我们评估了所有 Transformer 变体在自回归图像生成任务上的吞吐量。然而，另一个需要考虑的重要因素是延迟，即生成单张图像所需的总时间。为此，我们使用批大小 1，测量所有方法生成单张图像所需的时间。除了在 GPU 上运行推理外，我们还评估了在 CPU 上所需的时间。结果报告在表 5 中。

方法	秒 (CPU)		秒 (GPU)	
Softmax	72.6	(13.2×)	10.2	(1.4×)
Stateful-softmax	7.4	(1.3×)	10.4	(1.42×)
LSH-1	46.0	(8.3×)	19.2	(2.6×)
LSH-4	112.0	(20×)	55.8	(7.6×)
Linear (ours)	**5.5**	(1×)	**7.3**	(1×)

(a) MNIST 图像生成

方法	秒 (CPU)		秒 (GPU)	
Softmax	8651.4	(191.8×)	300.1	(4.9×)
Stateful-softmax	71.9	(1.6×)	70.4	(1.14×)
LSH-1	2318.9	(51.4×)	221.6	(3.6×)
LSH-4	5263.7	(116.7×)	683.9	(11.1×)
Linear (ours)	**45.1**	(1×)	**61.3**	(1×)

(b) CIFAR-10 图像生成

Table 5: 使用自回归 Transformer 在 MNIST 和 CIFAR-10 上生成单张图像所需时间比较。所有方法以批大小 1 在 CPU 和 GPU 上运行，报告总时间（秒）。越小越好。

表 5 解读

这个实验测量的是“生成一张图要多少秒”——批大小为1，纯粹看延迟。
MNIST：Linear 在 CPU 上只需 5.5 秒，GPU 上 7.3 秒。Softmax 在 GPU 上需要 10.2 秒，CPU 上需 72.6 秒。
CIFAR-10：差距更惊人。Softmax 在 CPU 上需要 8651 秒（约 2.4 小时！），GPU 上 300 秒（5 分钟）。Linear 只需 CPU 45 秒，GPU 61 秒。
有趣的观察：Linear 是唯一一个在 CPU 上比 GPU 更快的方法！原因是：RNN 式推理的每步计算量很小（只有矩阵加法和向量-矩阵乘），主要瓶颈是不可避免的序列循环（for 循环），GPU 的并行优势发挥不出来，反而 CPU 的低延迟在此占优。

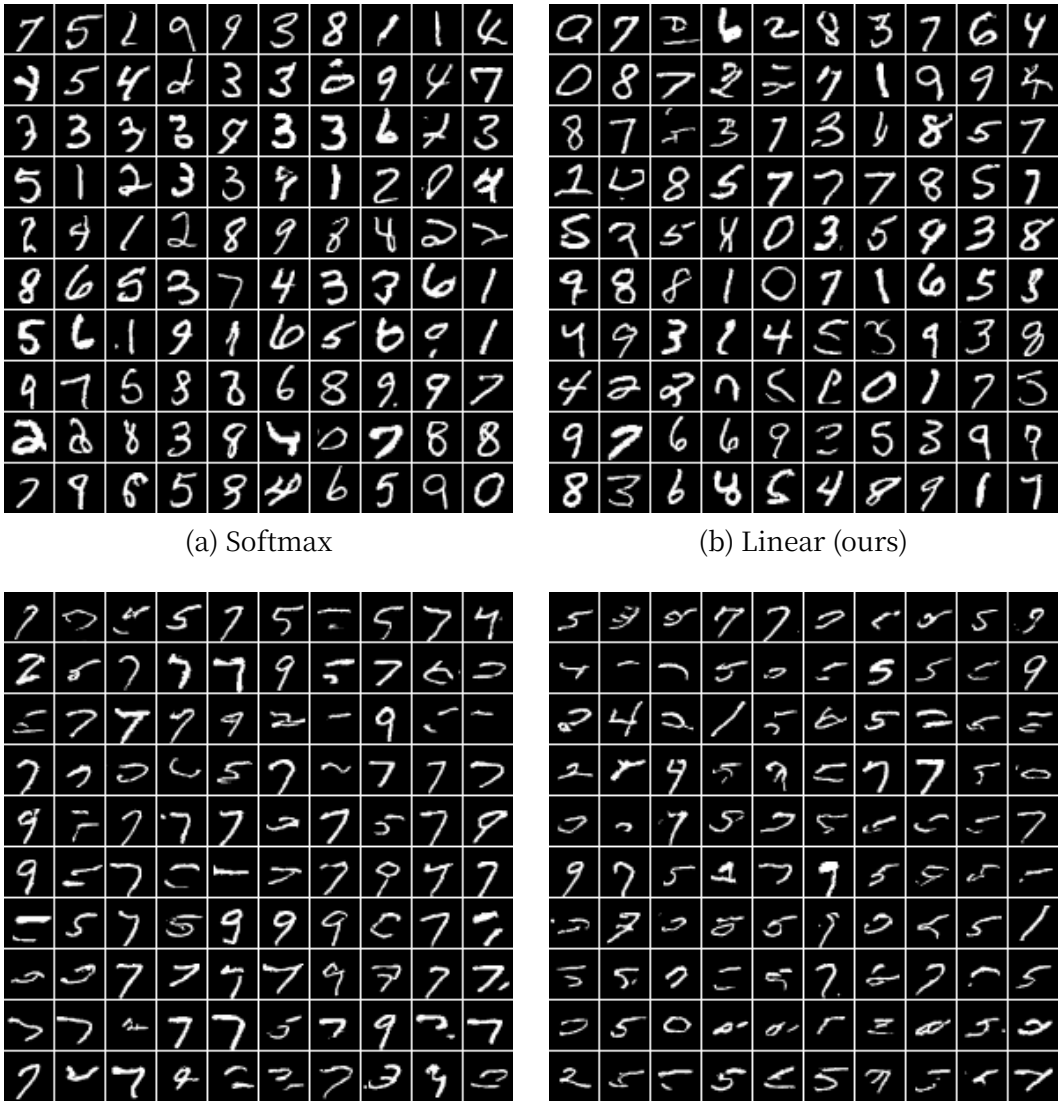
原文翻译

我们观察到所有方法都未充分利用 GPU，图像生成吞吐量远低于表 4 中所示。提出的线性 Transformer 比所有方法都快，特别是在 CIFAR-10 上生成图像时几乎比 softmax Transformer 快 6.6 倍。注意我们的线性自回归 Transformer 是唯一一种在每种情况下 CPU 都比 GPU 快的方法。这是因为将注意力作为 RNN 计算的成本如此之低，以至于主要的计算瓶颈变成了不可避免的序列外循环。

D 附录 D：图像生成定性结果

原文翻译

在本节中，我们提供图像生成实验的定性结果。由于所有模型的困惑度大致相同，如预期，定性差异并不显著。一个相当有趣的观察是 Reformer 模型在其无条件采样中提供的变化显著较少。此外，我们观察到图像补全是比无条件生成显著更容易的任务，因为所有模型表现都显著更好。



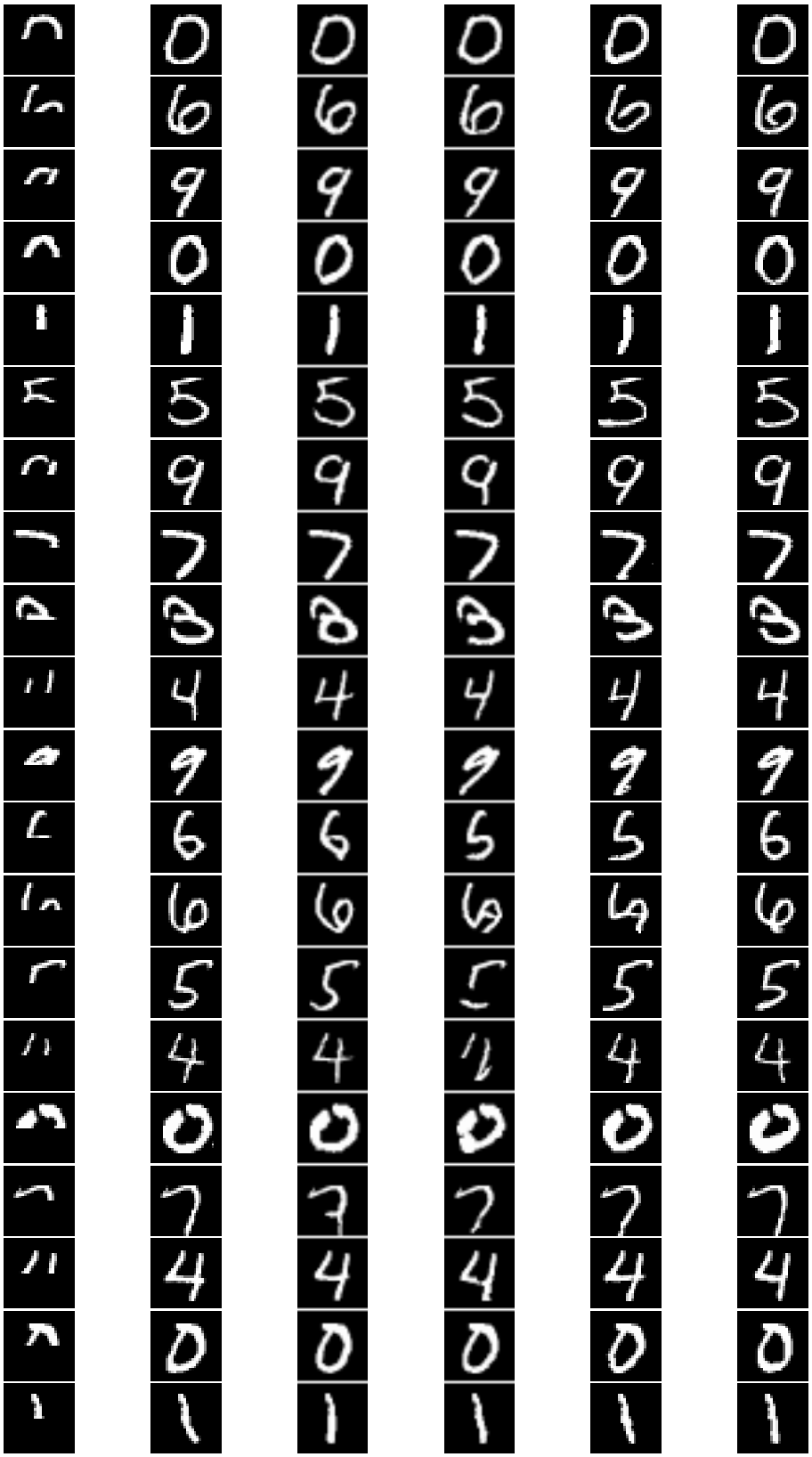
(a) Softmax (b) Linear (ours)

(c) LSH-1 (d) LSH-4

Figure 6: MNIST 数据集上各 Transformer 模型的无条件采样结果。

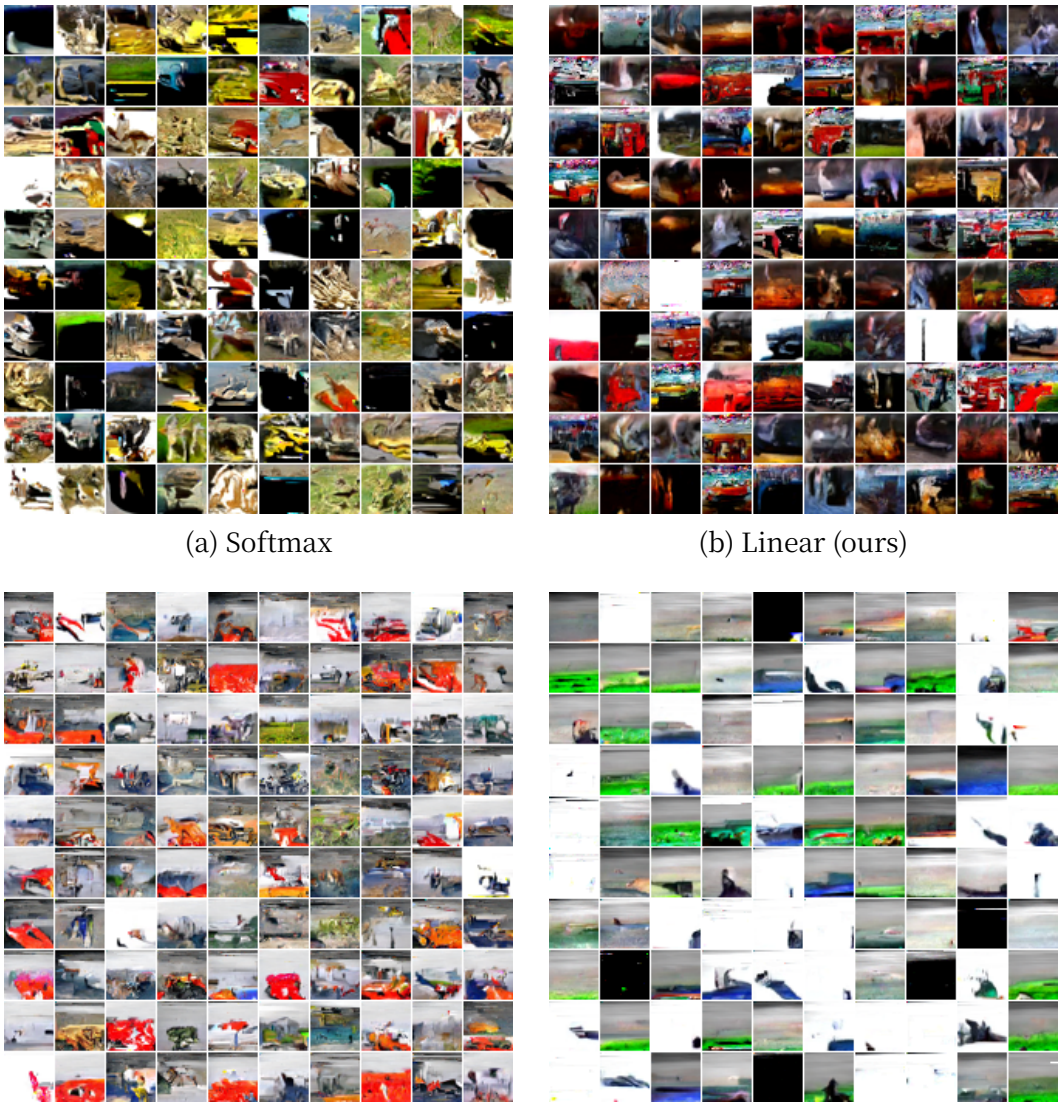
图 6 详细解读

四个子图分别展示了四种方法从零开始无条件生成的 MNIST 手写数字。
Softmax vs Linear：两者生成的数字质量非常接近，数字清晰、多样、笔画自然。这与表 1 中两者相似的 bits/dim 一致。
LSH 变体：样本多样性明显较低。特别是 LSH-4 的样本中出现了更多重复的模式，可能是 LSH 哈希分桶导致的信息瓶颈。



(a) 遮挡 (b) Softmax (c) Linear (d) LSH-1 (e) LSH-4 (f) 原图

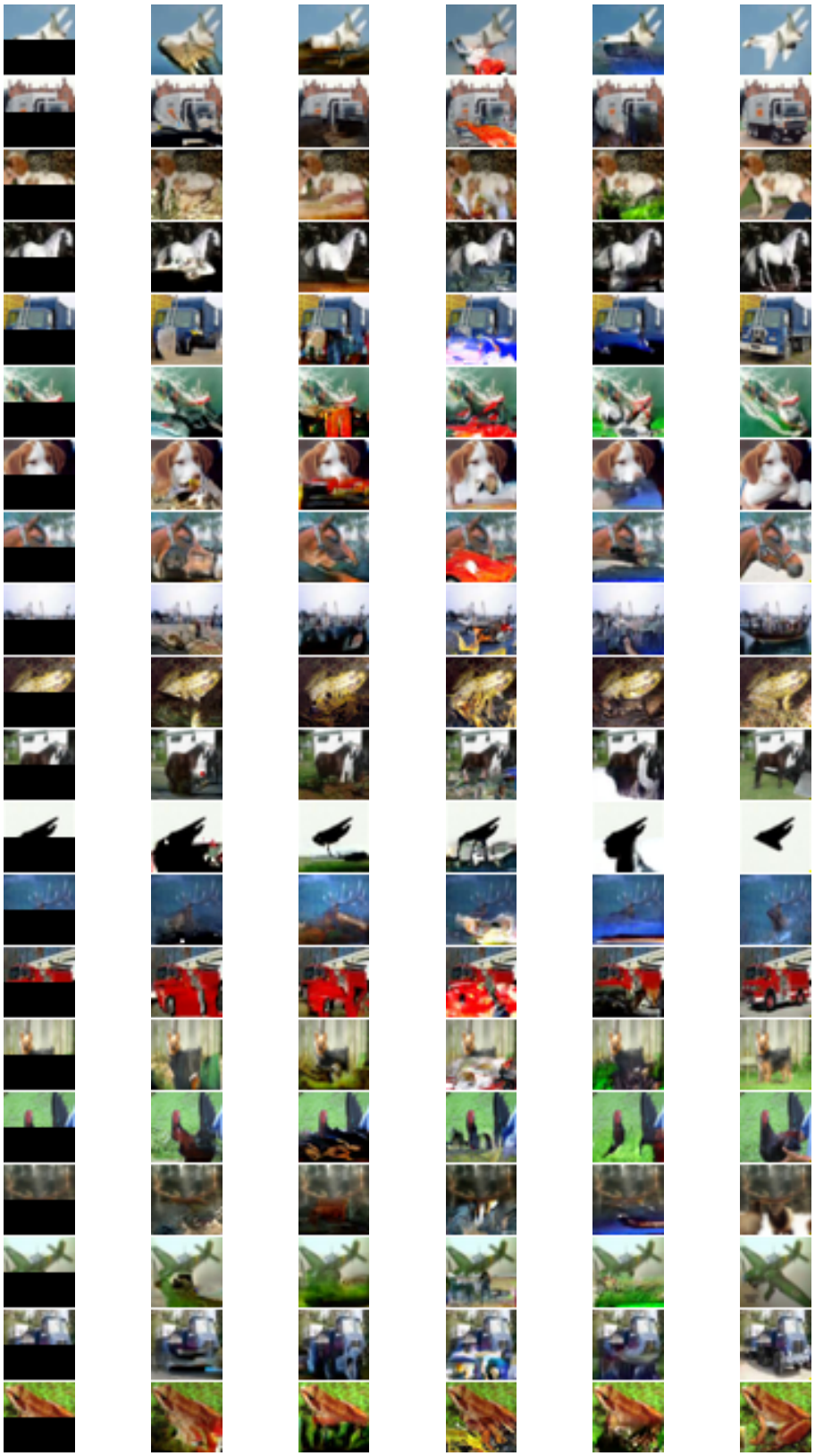
Figure 7: 所有训练模型的 MNIST 数字补全结果。



(a) Softmax (b) Linear (ours)

(c) LSH-1 (d) LSH-4

Figure 8: CIFAR-10 数据集上各 Transformer 模型的无条件采样结果。



(a) 遮挡 (b) Softmax (c) Linear (d) LSH-1 (e) LSH-4 (f) 原图

Figure 9: 所有训练模型的 CIFAR-10 图像补全结果。

CIFAR-10 定性结果解读

无条件生成：CIFAR-10 (32×32 彩色图像) 的无条件生成比 MNIST 困难得多。所有模型生成的图像都比较模糊，但可以看到颜色和空间结构的一致性。Softmax 和 Linear 的样本质量相近，都优于 LSH 变体。
图像补全：给定图像上半部分（如动物的头部、车辆的上部），模型需要补全下半部分。所有方法都能较好地保持颜色和结构的一致性，补全结果的图像类别大多可以识别。图像补全显著容易于无条件生成，因为已有一半的视觉信息作为条件。

E 附录 E：参考文献一览表

文献	主要内容	在本文中的引用原因
Vaswani et al. 2017 (Transformer)	提出 Transformer 架构和多头自注意力机制	本文工作的基础架构
Bahdanau et al. 2015	引入注意力机制的神经机器翻译	Transformer 的历史背景
Sutskever et al. 2014	序列到序列学习（Seq2Seq）	机器翻译的基础工作
Devlin et al. 2019 (BERT)	双向 Transformer 预训练	说明 Transformer 在 NLP 的成功
Sperber et al. 2018	自注意力声学模型	说明 Transformer 在语音的应用
Parmar et al. 2019	独立自注意力的视觉模型	说明 Transformer 在视觉的应用
Radford et al. 2018 (GPT-1)	生成式预训练语言模型	自回归预训练的代表
Radford et al. 2019 (GPT-2)	语言模型是无监督多任务学习者	自回归生成的成功案例
Yang et al. 2019 (XLNet)	广义自回归预训练	掩码语言建模的变体
Song et al. 2019 (MASS)	掩码序列到序列预训练	掩码预训练方法之一
Liu et al. 2020 (RoBERTa)	鲁棒优化的 BERT 预训练	掩码预训练方法之一
Dai et al. 2019 (Transformer-XL)	超越固定长度上下文的注意力语言模型	扩展 Transformer 上下文的方法
Child et al. 2019 (Sparse Transformer)	注意力矩阵的稀疏因式分解	降低注意力复杂度到 $O(N\sqrt{N})$
Kitaev et al. 2020 (Reformer)	基于 LSH 的高效 Transformer	主要基线，复杂度 $O(N \log N)$
Michel et al. 2019	16 个注意力头是否真的比 1 个好	权重剪枝方法
Lan et al. 2020 (ALBERT)	轻量级 BERT	权重分解方法
Zafrir et al. 2019 (Q8BERT)	8 位量化 BERT	权重量化方法

文献	主要内容	在本文中的引用原因
Clark et al. 2020 (ELECTRA)	替代 token 检测预训练	高效预训练目标
Lample et al. 2019	乘积键大内存层	增加层容量的方法
Sukhbaatar et al. 2019	自适应注意力跨度	扩展上下文的方法
Tsai et al. 2019	通过核的视角解析 Transformer 注意力	核函数视角的理论基础
Cordonnier et al. 2020	自注意力与卷积层的关系	理解自注意力的理论工作
Goodman 2001	快速最大熵训练的类别方法	大类别 softmax 加速的早期工作
Morin & Bengio 2005	层次概率神经网络语言模型	层次 softmax
Mnih & Hinton 2009	可扩展的层次分布式语言模型	softmax 瓶颈的历史背景
Blanc & Rendle 2017	基于核采样的自适应 softmax	用特征映射近似 softmax
Rawat et al. 2019	随机傅里叶特征采样 softmax	用特征映射近似 softmax
Shen et al. 2020	线性复杂度的高效注意力	同期工作，用于目标检测
Dehghani et al. 2018 (Universal Transformer)	在深度方向循环的 Transformer	区分深度循环与时间循环
Hochreiter & Schmidhuber 1997 (LSTM)	长短期记忆网络	RNN 基线和理论对比
Clevert et al. 2015	ELU 激活函数	特征映射中使用 ELU
LeCun et al. 2010	MNIST 手写数字数据集	图像生成实验的数据集
Krizhevsky et al. 2009	CIFAR-10 数据集	图像生成实验的数据集
Salimans et al. 2017 (PixelCNN++)	离散化 logistic 混合建模	输出分布的建模方式
Liu et al. 2019 (RAdam)	带预热的 Adam 优化器变体	训练中使用的优化器
Kingma & Ba 2014 (Adam)	Adam 优化器	语音识别实验中使用
Graves et al. 2006 (CTC)	连接时序分类	语音识别的损失函数
Paul & Baker 1992 (WSJ)	华尔街日报语音数据集	语音识别实验的数据集
Paszke et al. 2019 (PyTorch)	PyTorch 深度学习库	实现框架